
THE COMPLETE GUIDE

Node.js

Express.js

MongoDB

Interview Preparation Guide

From Scratch to Associate Software Engineer

Covering L1 & L2 Interview Levels

31

Chapters

12
5+

Interview Questions

2026 EDITION

Preface

Welcome to the **Node.js, Express.js & MongoDB Interview Preparation Guide**. This book is designed as a single resource that takes you from foundational concepts to the depth required for Associate Software Engineer interviews at L1 and L2 levels.

The MERN stack — minus React on the frontend — powers a huge proportion of modern backend systems. Companies hiring for Junior and Associate Backend / Full-Stack Engineer roles consistently test on these three technologies. Mastering them gives you a strong foundation for almost any Node.js role.

How This Book Is Organized

The book is divided into four parts:

Part	Title	Chapters
I	Node.js Fundamentals to Advanced	1 – 11
II	Express.js — Building Web APIs	12 – 21
III	MongoDB — From Basics to Performance	22 – 31
IV	Interview Questions & Answers (L1 & L2)	125+ Q&A

How to Use This Guide

If you are **new to Node.js / Express / MongoDB**, read Parts I-III sequentially. Build a small project alongside (a REST API with auth would be ideal). Then tackle the interview questions in Part IV.

If you **already have experience** and are preparing for interviews, skim Parts I-III to fill knowledge gaps and then focus heavily on Part IV. Aim to answer each L1 question without looking, and to discuss each L2 question with depth and trade-offs.

Every chapter contains code examples — **run them**. Reading alone won't prepare you for the live coding portions of interviews. Set up a local Node project and a MongoDB Atlas free cluster, and practice each pattern hands-on.

Conventions Used

Code blocks	Practical, runnable snippets you can copy and try.
Tables	Reference material — operators, status codes, comparisons.
NOTE boxes	Important context or clarifications.
TIP boxes	Best practices and pro suggestions.
WARNING boxes	Pitfalls to avoid.
IMPORTANT boxes	Topics that come up frequently in interviews.

A note on currency: the JavaScript and Node.js ecosystem moves fast. Where versions matter, this guide reflects Node.js LTS, Express 4.x/5.x and MongoDB 6.x/7.x conventions. Always cross-check with the official documentation for breaking changes.

Wishing you success in your interviews.
— Your study companion

Table of Contents

■ **PART I: Node.js Fundamentals to Advanced**

Chapter 1: Introduction to Node.js

Chapter 2: Modules in Node.js (CommonJS & ES Modules)

Chapter 3: NPM and package.json

Chapter 4: Core Modules (fs, path, http, os, crypto)

Chapter 5: The Event Loop

Chapter 6: Asynchronous Programming (Callbacks, Promises, async/await)

Chapter 7: Events and EventEmitter

Chapter 8: Streams and Buffers

Chapter 9: Error Handling

Chapter 10: Process and Child Processes

Chapter 11: Best Practices

■ **PART II: Express.js — Building Web APIs**

Chapter 12: Introduction to Express.js

Chapter 13: Routing

Chapter 14: Middleware

Chapter 15: Request and Response Objects

Chapter 16: REST API Development

Chapter 17: Authentication & Authorization

Chapter 18: Request Validation

Chapter 19: File Uploads

Chapter 20: Security Best Practices

Chapter 21: Testing Express Applications

■ **PART III: MongoDB — From Basics to Performance**

Chapter 22: Introduction to MongoDB

Chapter 23: CRUD Operations

Chapter 24: Advanced Queries

Chapter 25: Indexes

Chapter 26: Aggregation Framework

Chapter 27: Schema Design Patterns

Chapter 28: Mongoose ODM

Chapter 29: Transactions

Chapter 30: Replication and Sharding

Chapter 31: Performance Optimization

■ **PART IV: Interview Questions & Answers**

Node.js — L1 (Fundamental Questions, Q1–Q25)
Node.js — L2 (Advanced Questions, Q26–Q42)
Express.js — L1 (Fundamental Questions, Q43–Q60)
Express.js — L2 (Advanced Questions, Q61–Q73)
MongoDB — L1 (Fundamental Questions, Q74–Q94)
MongoDB — L2 (Advanced Questions, Q95–Q110)
Coding & Scenario Questions (Q111–Q125)
Final Interview Tips

PART I

Node.js

PART I

Node.js

Chapter 1: Introduction to Node.js

1.1 What is Node.js?

Node.js is an open-source, cross-platform JavaScript runtime environment that executes JavaScript code outside of a web browser. It was created by Ryan Dahl in 2009 and is built on top of the Google V8 JavaScript engine (the same engine that powers Google Chrome). Before Node.js, JavaScript was confined to running inside browsers. Node.js liberated JavaScript and allowed developers to build full-stack applications using a single programming language.

Node.js is designed to build scalable network applications. It uses an event-driven, non-blocking I/O model that makes it lightweight and efficient, perfect for data-intensive real-time applications that run across distributed devices.

1.2 Why Node.js?

- **JavaScript Everywhere:** Use the same language on both client and server, reducing context switching for developers.
- **Non-Blocking I/O:** Handles many concurrent connections efficiently without creating a thread for each one.
- **NPM Ecosystem:** The world's largest software registry with over two million packages.
- **Fast Execution:** V8 compiles JavaScript directly to native machine code for high performance.
- **Active Community:** Massive community contributing libraries, tools, and frameworks.
- **Scalability:** Built for handling thousands of concurrent connections with minimal overhead.
- **Real-Time Apps:** Excellent for chat applications, gaming servers, collaboration tools, and live streaming.

1.3 Node.js Architecture

Node.js follows a Single-Threaded Event Loop architecture but it can handle many client requests concurrently. Understanding its architecture is fundamental for interview preparation.

Key Components

Component	Description
V8 Engine	Google's open-source JavaScript engine written in C++. Compiles JS to native machine code.
libuv	C library that provides the event loop, async I/O, thread pool, and cross-platform abstractions.
Event Loop	The mechanism that allows Node.js to perform non-blocking I/O operations on a single thread.
Thread Pool	A pool of worker threads (default size 4) used by libuv for expensive operations like file system access.
Node APIs	Built-in JavaScript libraries for file system, networking, streams, etc.
C++ Bindings	Bridge between JavaScript and C/C++ libraries that perform low-level operations.

1.4 Installing Node.js

Node.js can be installed in several ways. The most common methods include:

- **Official Installer:** Download from nodejs.org and run the installer for your OS.
- **Package Manager:** Use apt (Ubuntu), brew (macOS), or chocolatey (Windows).
- **nvm (Node Version Manager):** Recommended for developers who work with multiple Node.js versions.

```
# Install nvm and use the latest LTS version of Node.js
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
nvm install --lts
nvm use --lts

# Verify installation
node --version
npm --version
```

1.5 Your First Node.js Program

Create a file named `app.js`:

```
// app.js
console.log('Hello, World!');

// Variables and template literals
const name = 'Developer';
console.log(`Welcome, ${name}!`);

// A simple function
function greet(person) {
  return `Hello, ${person}!`;
}
console.log(greet('Node.js'));
```

Run with:

```
node app.js
```

Chapter 2: Modules in Node.js

2.1 What are Modules?

A module in Node.js is a reusable block of code whose existence does not accidentally impact other code. Modules help us organize code into manageable, maintainable pieces. Node.js has a simple module loading system where files and modules are in one-to-one correspondence.

2.2 Types of Modules

Module Type	Description & Examples
Core Modules	Built into Node.js (fs, http, path, url, events, stream, crypto, os).
Local Modules	Modules created locally within your project, exported from files you write.
Third-Party Modules	Installed via npm from the npm registry (express, mongoose, lodash, etc.).

2.3 CommonJS Modules

CommonJS is the original module system in Node.js. It uses `require()` to import and `module.exports` to export.

```
// math.js – exporting from a module
function add(a, b) {
  return a + b;
}
function subtract(a, b) {
  return a - b;
}
const PI = 3.14159;

// Export multiple items
module.exports = { add, subtract, PI };

// Alternative single export
// module.exports = add;

// Named exports
// exports.add = add;
// exports.subtract = subtract;
```

```
// app.js – importing from a module
const math = require('./math');
console.log(math.add(5, 3));      // 8
console.log(math.subtract(10, 4)); // 6
console.log(math.PI);            // 3.14159

// Destructured import
const { add, PI } = require('./math');
console.log(add(2, 3)); // 5
```

2.4 ES Modules (ESM)

ES Modules are the official JavaScript standard for modules. Node.js supports them when you use the `.mjs` extension or set `"type": "module"` in `package.json`.

```
// math.mjs
export function add(a, b) {
  return a + b;
}
export const PI = 3.14159;
export default function multiply(a, b) {
  return a * b;
}

// app.mjs
import multiply, { add, PI } from './math.mjs';
console.log(add(2, 3));      // 5
console.log(multiply(4, 5)); // 20
console.log(PI);            // 3.14159
```

2.5 CommonJS vs ES Modules

Feature	CommonJS	ES Modules
Syntax	require / module.exports	import / export
Loading	Synchronous	Asynchronous
File extension	.js (default)	.mjs or "type": "module"
Top-level await	Not supported	Supported
Tree shaking	Not effective	Effective
Browser support	Needs bundler	Native support
this binding	this = module.exports	this = undefined

2.6 The Module Wrapper Function

Before a module's code is executed, Node.js wraps it in a function. This is why each module has its own scope and special variables like `module`, `exports`, `__filename`, and `__dirname` are available.

```
// Conceptually, every module is wrapped like this:
(function(exports, require, module, __filename, __dirname) {
  // Your module code lives here
});
```

NOTE: `__dirname` gives the absolute path of the directory containing the current file.
`__filename` gives the absolute path including the file name. These are not available in ES Modules — use `import.meta.url` instead.

Chapter 3: NPM and Package Management

3.1 What is NPM?

NPM (Node Package Manager) is the default package manager for Node.js. It consists of three components: a website where you can browse packages, a registry that stores packages, and a command-line interface used by developers to install and manage packages. NPM is the world's largest software registry with over two million packages available.

3.2 package.json

The `package.json` file is the heart of any Node.js project. It contains metadata about your project, dependencies, scripts, and configuration.

```
{
  "name": "my-app",
  "version": "1.0.0",
  "description": "A sample Node.js application",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js",
    "test": "jest"
  },
  "dependencies": {
    "express": "^4.18.2",
    "mongoose": "^7.0.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.1",
    "jest": "^29.0.0"
  },
  "engines": {
    "node": ">=18.0.0"
  },
  "author": "Your Name",
  "license": "MIT"
}
```

3.3 Common NPM Commands

Command	Description
npm init	Initialize a new project (creates package.json)
npm init -y	Initialize with default values
npm install <pkg>	Install a package and add to dependencies
npm install <pkg> --save-dev	Install as a dev dependency
npm install -g <pkg>	Install package globally
npm uninstall <pkg>	Remove a package
npm update	Update packages to latest versions (per semver)
npm list	List installed packages
npm outdated	Check for outdated packages
npm run <script>	Run a custom script defined in package.json
npm audit	Scan for known vulnerabilities
npm ci	Clean install (uses package-lock.json strictly)

3.4 Semantic Versioning (semver)

NPM uses Semantic Versioning to manage package versions. A version number has three parts: MAJOR.MINOR.PATCH (e.g., 4.18.2).

- **MAJOR (4):** Incremented for breaking changes.
- **MINOR (18):** Incremented for new backwards-compatible features.
- **PATCH (2):** Incremented for backwards-compatible bug fixes.

Version Range Specifiers

Specifier	Meaning	Example
<code>^4.18.2</code>	Caret: allows minor and patch updates	4.18.2 to <5.0.0
<code>~4.18.2</code>	Tilde: allows only patch updates	4.18.2 to <4.19.0
<code>4.18.2</code>	Exact version	Only 4.18.2
<code>*</code>	Any version	Latest
<code>>=4.18.0</code>	Greater than or equal to	4.18.0 and above
<code>4.x</code>	Wildcard for any 4.x.x	4.0.0 to <5.0.0

3.5 package-lock.json

`package-lock.json` is automatically generated when you run `npm install`. It records the exact version of every installed package and its dependencies, ensuring that future installations produce the same dependency tree across different environments. Always commit this file to version control.

Chapter 4: Core Modules

Node.js comes with many built-in modules that allow you to perform common tasks without installing any external packages. Let's explore the most important ones.

4.1 fs (File System)

The fs module provides an API for interacting with the file system. Almost all fs methods come in synchronous and asynchronous flavors.

Reading Files

```
const fs = require('fs');
const fsPromises = require('fs').promises;

// Asynchronous (callback-based)
fs.readFile('./data.txt', 'utf-8', (err, data) => {
  if (err) return console.error(err);
  console.log(data);
});

// Synchronous (blocks the event loop)
try {
  const data = fs.readFileSync('./data.txt', 'utf-8');
  console.log(data);
} catch (err) {
  console.error(err);
}

// Promise-based (modern approach)
async function readFile() {
  try {
    const data = await fsPromises.readFile('./data.txt', 'utf-8');
    console.log(data);
  } catch (err) {
    console.error(err);
  }
}
```

Writing and Appending Files

```
// Write (overwrites existing file)
fs.writeFile('./output.txt', 'Hello, World!', err => {
  if (err) return console.error(err);
  console.log('File written');
});

// Append to an existing file
fs.appendFile('./output.txt', '\nMore content', err => {
  if (err) return console.error(err);
});

// Delete a file
fs.unlink('./output.txt', err => {
  if (err) return console.error(err);
});
```

Working with Directories

```
// Create directory
fs.mkdir('./newDir', { recursive: true }, err => { /* ... */ });

// Read directory contents
fs.readdir('./', (err, files) => {
  if (err) return console.error(err);
  files.forEach(file => console.log(file));
});

// Get file/folder stats
fs.stat('./data.txt', (err, stats) => {
  console.log('Size:', stats.size);
  console.log('Is file?', stats.isFile());
  console.log('Created:', stats.birthtime);
});

// Remove directory
fs.rmdir('./newDir', { recursive: true }, err => { /* ... */ });
```

4.2 path

The `path` module provides utilities for working with file and directory paths. It handles differences between operating systems (forward vs. backward slashes) automatically.

```
const path = require('path');

// Join paths (uses correct separator for OS)
const fullPath = path.join(__dirname, 'data', 'users.json');
// /home/user/project/data/users.json

// Resolve to absolute path
const abs = path.resolve('./data/users.json');

// Get file extension
path.extname('file.txt'); // '.txt'

// Get base name (filename with extension)
path.basename('/foo/bar/baz.txt'); // 'baz.txt'
path.basename('/foo/bar/baz.txt', '.txt'); // 'baz'

// Get directory name
path.dirname('/foo/bar/baz.txt'); // '/foo/bar'

// Parse a path into its components
path.parse('/home/user/file.txt');
// { root: '/', dir: '/home/user', base: 'file.txt', ext: '.txt', name: 'file' }
```

4.3 http

The http module allows you to create HTTP servers and make HTTP requests. This is the foundation upon which Express.js is built.

```
const http = require('http');

const server = http.createServer((req, res) => {
  // Set the response header
  res.writeHead(200, { 'Content-Type': 'application/json' });

  // Routing based on URL
  if (req.url === '/') {
    res.end(JSON.stringify({ message: 'Welcome' }));
  } else if (req.url === '/users') {
    res.end(JSON.stringify({ users: ['Alice', 'Bob'] }));
  } else {
    res.writeHead(404);
    res.end(JSON.stringify({ error: 'Not Found' }));
  }
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000');
});
```

4.4 url

```
const url = require('url');

const myURL = new URL('https://example.com:8080/path?id=42&name=ben#section');
console.log(myURL.hostname);    // 'example.com'
console.log(myURL.port);        // '8080'
console.log(myURL.pathname);    // '/path'
console.log(myURL.search);      // '?id=42&name=ben'
console.log(myURL.searchParams.get('id')); // '42'
console.log(myURL.hash);        // '#section'
```

4.5 os

```
const os = require('os');

console.log(os.platform()); // 'linux', 'darwin', 'win32'
console.log(os.arch());     // 'x64', 'arm64'
console.log(os.cpus());     // Array of CPU info
console.log(os.totalmem()); // Total system memory (bytes)
console.log(os.freemem());  // Free memory (bytes)
console.log(os.homedir());  // User's home directory
console.log(os.hostname()); // Hostname of the machine
console.log(os.uptime());   // System uptime in seconds
```

4.6 crypto

```
const crypto = require('crypto');

// Generate a random secret token
const token = crypto.randomBytes(32).toString('hex');

// Hash a password (use bcrypt in production!)
const hash = crypto.createHash('sha256')
  .update('mypassword')
  .digest('hex');

// HMAC
const hmac = crypto.createHmac('sha256', 'secret-key')
  .update('important data')
  .digest('hex');

// Generate a UUID
const id = crypto.randomUUID();
```

Chapter 5: The Event Loop

5.1 What is the Event Loop?

The event loop is what allows Node.js to perform non-blocking I/O operations — despite the fact that JavaScript is single-threaded — by offloading operations to the system kernel whenever possible. This is one of the most important and frequently asked interview topics.

When Node.js starts, it initializes the event loop, processes the provided input script, then begins processing the event loop. The event loop has several phases, each with its own queue of callbacks to execute.

5.2 Phases of the Event Loop

Phase	Description
Timers	Executes callbacks scheduled by <code>setTimeout()</code> and <code>setInterval()</code> .
Pending callbacks	Executes I/O callbacks deferred to the next iteration.
Idle, prepare	Used internally by Node.js.
Poll	Retrieves new I/O events and executes their callbacks. Blocks here if appropriate.
Check	Executes <code>setImmediate()</code> callbacks.
Close callbacks	Executes close event callbacks like <code>socket.on('close', ...)</code> .

5.3 Microtasks vs Macrotasks

Between each phase of the event loop, Node.js processes microtasks. There are two microtask queues:

- **process.nextTick queue:** Highest priority. Always processed before any other microtasks.
- **Promise queue:** Resolved/rejected promise callbacks (`then`, `catch`, `finally`).

```
console.log('1');

setTimeout(() => console.log('2'), 0);
setImmediate(() => console.log('3'));

Promise.resolve().then(() => console.log('4'));
process.nextTick(() => console.log('5'));

console.log('6');

// Output: 1, 6, 5, 4, 2, 3
// 1 and 6 – synchronous code first
// 5 – process.nextTick (highest priority microtask)
// 4 – Promise microtask
// 2 – setTimeout (Timers phase)
// 3 – setImmediate (Check phase)
```

5.4 setTimeout vs setImmediate vs process.nextTick

Function	When It Executes
process.nextTick(fn)	Immediately after the current operation completes, before any I/O events
Promise.then(fn)	After process.nextTick callbacks but before the next event loop phase
setTimeout(fn, 0)	In the next Timers phase (minimum delay ~1ms)
setImmediate(fn)	In the Check phase, after the Poll phase

TIP: Common interview trick: Inside an I/O callback, setImmediate will always execute before setTimeout(fn, 0). Outside of I/O callbacks, the order is not guaranteed.

5.5 Why is Node.js Single-Threaded?

Node.js operates on a single main thread that runs JavaScript code. However, it is not entirely single-threaded — it uses a thread pool (provided by libuv, default size 4) for expensive operations like file system access, DNS lookups, and some crypto operations. This design avoids the complexity of multi-threading (race conditions, deadlocks) while still providing excellent concurrent performance for I/O-heavy workloads.

Chapter 6: Asynchronous Programming

6.1 Synchronous vs Asynchronous

Synchronous code runs line by line; each operation must complete before the next one starts. This is also called blocking code.

Asynchronous code initiates an operation and immediately moves to the next instruction. When the operation completes, a callback, promise, or async function is notified. This is non-blocking code.

```
// Synchronous (blocking)
const fs = require('fs');
const data = fs.readFileSync('./large-file.txt', 'utf-8');
console.log(data);
console.log('This runs AFTER file is read');

// Asynchronous (non-blocking)
fs.readFile('./large-file.txt', 'utf-8', (err, data) => {
  console.log(data);
});
console.log('This runs BEFORE file is read');
```

6.2 Callbacks

A callback is a function passed as an argument to another function, to be executed once an asynchronous operation completes. Node.js follows the error-first callback pattern: the first argument is reserved for an error object (or null if successful).

```
function fetchUser(id, callback) {
  setTimeout(() => {
    if (id < 0) {
      return callback(new Error('Invalid id'));
    }
    callback(null, { id, name: 'John Doe' });
  }, 1000);
}

fetchUser(1, (err, user) => {
  if (err) return console.error(err);
  console.log(user);
});
```

Callback Hell

When you chain multiple asynchronous operations using callbacks, the code can quickly become difficult to read and maintain. This is often called "callback hell" or the "pyramid of doom."

```
// The Pyramid of Doom
getUser(1, (err, user) => {
  if (err) return console.error(err);
  getPosts(user.id, (err, posts) => {
    if (err) return console.error(err);
    getComments(posts[0].id, (err, comments) => {
      if (err) return console.error(err);
      getUserDetails(comments[0].userId, (err, details) => {
        if (err) return console.error(err);
        // Deeply nested and hard to read!
        console.log(details);
      });
    });
  });
});
```

6.3 Promises

A Promise is an object representing the eventual completion or failure of an asynchronous operation. It has three states: **pending** (initial state), **fulfilled** (operation completed successfully), and **rejected** (operation failed).

```
// Creating a Promise
const fetchUser = (id) => {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id < 0) {
        return reject(new Error('Invalid id'));
      }
      resolve({ id, name: 'John Doe' });
    }, 1000);
  });
};

// Consuming a Promise
fetchUser(1)
  .then(user => console.log(user))
  .catch(err => console.error(err))
  .finally(() => console.log('Done'));
```

Promise Chaining

```
// Promises eliminate callback hell with flat chaining
getUser(1)
  .then(user => getPosts(user.id))
  .then(posts => getComments(posts[0].id))
  .then(comments => getUserDetails(comments[0].userId))
  .then(details => console.log(details))
  .catch(err => console.error('Error:', err));
```

Promise Static Methods

Method	Description
Promise.all([p1, p2])	Waits for all promises to fulfill. Rejects fast if any rejects.
Promise.allSettled([p1, p2])	Waits for all promises to settle (fulfill or reject). Returns an array of results.
Promise.race([p1, p2])	Settles with whichever promise settles first.
Promise.any([p1, p2])	Resolves with the first fulfilled promise. Rejects only if all reject.
Promise.resolve(value)	Returns a promise that is already resolved with value.
Promise.reject(reason)	Returns a promise that is already rejected with reason.

```
// Promise.all - parallel execution
const [users, posts, comments] = await Promise.all([
  fetchUsers(),
  fetchPosts(),
  fetchComments()
]);

// Promise.allSettled - get all results regardless of success
const results = await Promise.allSettled([
  fetch('/api/a'),
  fetch('/api/b')
]);
results.forEach(r => {
  if (r.status === 'fulfilled') console.log(r.value);
  else console.error(r.reason);
});
```

6.4 Async / Await

Async/await is syntactic sugar built on top of promises that allows you to write asynchronous code that looks and behaves like synchronous code. An **async** function always returns a promise, and the **await** keyword pauses execution until the awaited promise settles.

```
async function loadDashboard() {
  try {
    const user = await getUser(1);
    const posts = await getPosts(user.id);
    const comments = await getComments(posts[0].id);
    const details = await getUserDetails(comments[0].userId);
    console.log(details);
    return details;
  } catch (err) {
    console.error('Error:', err);
    throw err; // Re-throw for caller to handle
  }
}

// Calling an async function
loadDashboard()
  .then(data => console.log('Loaded:', data))
  .catch(err => console.error(err));
```

Parallel Execution with async/await

```
// SEQUENTIAL – slow (takes time of all combined)
async function sequential() {
  const a = await fetchA(); // 1s
  const b = await fetchB(); // 1s
  const c = await fetchC(); // 1s
  return [a, b, c];          // Total: ~3s
}

// PARALLEL – fast (takes time of slowest)
async function parallel() {
  const [a, b, c] = await Promise.all([
    fetchA(), // All start immediately
    fetchB(),
    fetchC()
  ]);
  return [a, b, c];          // Total: ~1s
}
```

TIP: When multiple async operations are independent of each other, use `Promise.all()` instead of awaiting them sequentially. This is a common interview question and performance pitfall.

Chapter 7: Events and EventEmitter

7.1 The Event-Driven Architecture

Node.js is built around an event-driven architecture where certain objects (called "emitters") emit named events that cause function objects ("listeners") to be called. Many of Node.js's core APIs are built upon this pattern.

7.2 Using EventEmitter

```
const EventEmitter = require('events');

class OrderEmitter extends EventEmitter {}
const orderEvents = new OrderEmitter();

// Register a listener
orderEvents.on('orderPlaced', (order) => {
  console.log(`Order ${order.id} received for $$${order.total}`);
});

// One-time listener (auto-removed after first call)
orderEvents.once('serverStarted', () => {
  console.log('Server is up and running');
});

// Emit an event
orderEvents.emit('orderPlaced', { id: 101, total: 49.99 });

// Remove a specific listener
function handler() { /* ... */ }
orderEvents.on('test', handler);
orderEvents.off('test', handler);

// Get listener count
console.log(orderEvents.listenerCount('orderPlaced'));
```

7.3 EventEmitter Methods

Method	Description
<code>on(event, listener)</code>	Add a listener for the event
<code>once(event, listener)</code>	Add a listener that fires only once
<code>emit(event, ...args)</code>	Emit an event with optional arguments
<code>off(event, listener)</code>	Remove a specific listener (alias for <code>removeListener</code>)
<code>removeAllListeners(event)</code>	Remove all listeners for an event
<code>listenerCount(event)</code>	Get the number of listeners for an event
<code>setMaxListeners(n)</code>	Set the maximum number of listeners (default 10)

Chapter 8: Streams and Buffers

8.1 What are Streams?

Streams are objects that let you read data from a source or write data to a destination in a continuous fashion. Streams are useful when working with large amounts of data because they let you process data piece by piece rather than loading the entire dataset into memory.

8.2 Types of Streams

Type	Description	Example
Readable	Read data from a source	<code>fs.createReadStream()</code>
Writable	Write data to a destination	<code>fs.createWriteStream()</code>
Duplex	Both readable and writable	<code>net.Socket</code>
Transform	Duplex stream that modifies data	<code>zlib.createGzip()</code>

8.3 Using Streams

```
const fs = require('fs');

// Reading with streams (memory-efficient for large files)
const readStream = fs.createReadStream('./large-file.txt', {
  encoding: 'utf-8',
  highWaterMark: 64 * 1024 // 64KB chunks
});

readStream.on('data', chunk => {
  console.log('Got chunk of size:', chunk.length);
});
readStream.on('end', () => console.log('Done'));
readStream.on('error', err => console.error(err));

// Writing with streams
const writeStream = fs.createWriteStream('./output.txt');
writeStream.write('Hello\n');
writeStream.write('World\n');
writeStream.end();
```

8.4 Pipe and Pipeline

`pipe()` connects the output of a readable stream to the input of a writable stream. `pipeline()` is the modern, preferred way to chain streams because it handles errors automatically and cleans up resources.

```
const fs = require('fs');
const zlib = require('zlib');
const { pipeline } = require('stream/promises');

// Using pipe (older approach)
fs.createReadStream('input.txt')
  .pipe(zlib.createGzip())
  .pipe(fs.createWriteStream('input.txt.gz'));

// Using pipeline (preferred – better error handling)
async function compress() {
  try {
    await pipeline(
      fs.createReadStream('input.txt'),
      zlib.createGzip(),
      fs.createWriteStream('input.txt.gz')
    );
    console.log('Compression complete');
  } catch (err) {
    console.error('Pipeline failed:', err);
  }
}
compress();
```

8.5 Buffers

A Buffer is a fixed-size chunk of memory that holds raw binary data. Buffers are used when dealing with binary streams of data such as TCP streams, file system operations, or image data. The Buffer class is a global, so you don't need to require it.

```
// Create a buffer from a string
const buf1 = Buffer.from('Hello, World!', 'utf-8');
console.log(buf1); // <Buffer 48 65 6c 6c 6f 2c 20 ...>
console.log(buf1.length); // 13
console.log(buf1.toString()); // 'Hello, World!'

// Allocate a buffer of specific size
const buf2 = Buffer.alloc(10); // Zero-filled (safe)
const buf3 = Buffer.allocUnsafe(10); // Uninitialized (faster, but unsafe)

// Write to a buffer
buf2.write('Hello');
console.log(buf2.toString()); // 'Hello\x00\x00\x00\x00\x00'

// Concatenate buffers
const combined = Buffer.concat([buf1, buf2]);
```

Chapter 9: Error Handling

9.1 Types of Errors

- **Standard JavaScript Errors:** EvalError, SyntaxError, RangeError, ReferenceError, TypeError, URIError.
- **System Errors:** Triggered when something happens at the OS level (e.g., ENOENT for missing file).
- **User-Specified Errors:** Custom errors created by extending the Error class.
- **Assertion Errors:** Thrown by the assert module when a condition is false.

9.2 try / catch / finally

```
try {  
    const data = JSON.parse('{invalid json}');  
} catch (err) {  
    console.error('Parse error:', err.message);  
} finally {  
    console.log('Cleanup code runs regardless of success/failure');  
}
```

9.3 Custom Error Classes

```
class ValidationError extends Error {
  constructor(message, field) {
    super(message);
    this.name = 'ValidationError';
    this.field = field;
    this.statusCode = 400;
  }
}

class NotFoundError extends Error {
  constructor(resource) {
    super(`${resource} not found`);
    this.name = 'NotFoundError';
    this.statusCode = 404;
  }
}

function getUser(id) {
  if (!id) throw new ValidationError('id is required', 'id');
  if (id === 999) throw new NotFoundError('User');
  return { id, name: 'John' };
}

try {
  getUser(999);
} catch (err) {
  if (err instanceof ValidationError) console.log('Validation:', err.field);
  else if (err instanceof NotFoundError) console.log('Not found');
  else throw err;
}
```

9.4 Uncaught Exceptions and Rejections

```
// Catch errors not handled anywhere else (last resort)
process.on('uncaughtException', (err) => {
  console.error('Uncaught Exception:', err);
  // Log error, then exit – process is in an undefined state
  process.exit(1);
});

process.on('unhandledRejection', (reason, promise) => {
  console.error('Unhandled Rejection at:', promise);
  console.error('Reason:', reason);
  process.exit(1);
});
```

WARNING: Never use `uncaughtException` as a routine error-handling mechanism. After an uncaught exception, the application is in an undefined state and should be restarted. Always handle errors at the source.

Chapter 10: Process and Child Processes

10.1 The Process Object

process is a global object that provides information about, and control over, the current Node.js process. It's an instance of EventEmitter.

```
// Process information
console.log(process.pid);      // Process ID
console.log(process.platform); // 'linux', 'darwin', 'win32'
console.log(process.version);  // Node.js version
console.log(process.cwd());    // Current working directory
console.log(process.env);      // Environment variables
console.log(process.argv);     // Command-line arguments
console.log(process.uptime()); // Seconds since start
console.log(process.memoryUsage()); // Memory usage stats

// Exiting the process
process.exit(0); // Success
process.exit(1); // Failure
```

10.2 Environment Variables

```
// Access environment variables
const PORT = process.env.PORT || 3000;
const DB_URL = process.env.DATABASE_URL;
const NODE_ENV = process.env.NODE_ENV || 'development';

// Using dotenv for .env files
require('dotenv').config();

// .env file:
// PORT=3000
// DATABASE_URL=mongodb://localhost/mydb
// JWT_SECRET=supersecret
```

10.3 child_process Module

The `child_process` module lets you spawn subprocesses. This is useful for running CPU-intensive tasks without blocking the main event loop, or for running other programs (like git, ffmpeg).

```
const { exec, spawn, fork } = require('child_process');

// exec – runs a command in a shell and buffers output
exec('ls -la', (err, stdout, stderr) => {
  if (err) return console.error(err);
  console.log(stdout);
});

// spawn – streams data, best for long-running processes
const ls = spawn('ls', ['-la']);
ls.stdout.on('data', data => console.log(`stdout: ${data}`));
ls.stderr.on('data', data => console.error(`stderr: ${data}`));
ls.on('close', code => console.log(`Exited with code ${code}`));

// fork – spawns a new Node.js process with IPC channel
const child = fork('./worker.js');
child.send({ task: 'compute', data: [1, 2, 3] });
child.on('message', msg => console.log('From child:', msg));
```

10.4 Cluster Module

The cluster module allows you to create child processes (workers) that share server ports. This lets you take advantage of multi-core systems since Node.js's main thread is single-threaded.

```
const cluster = require('cluster');
const http = require('http');
const os = require('os');

if (cluster.isPrimary) {
  const numCPUs = os.cpus().length;
  console.log(`Primary ${process.pid} is running`);
  for (let i = 0; i < numCPUs; i++) {
    cluster.fork();
  }
  cluster.on('exit', (worker) => {
    console.log(`Worker ${worker.process.pid} died, restarting...`);
    cluster.fork();
  });
} else {
  http.createServer((req, res) => {
    res.writeHead(200);
    res.end(`Handled by worker ${process.pid}`);
  }).listen(3000);
  console.log(`Worker ${process.pid} started`);
}
```

10.5 Worker Threads

Worker threads enable true multi-threading in Node.js for CPU-intensive JavaScript operations. Unlike `child_process`, worker threads share memory via `SharedArrayBuffer` and have less overhead.

```
// main.js
const { Worker } = require('worker_threads');
const worker = new Worker('./worker.js', { workerData: 1000000 });
worker.on('message', result => console.log('Result:', result));
worker.on('error', err => console.error(err));

// worker.js
const { parentPort, workerData } = require('worker_threads');
let sum = 0;
for (let i = 0; i <= workerData; i++) sum += i;
parentPort.postMessage(sum);
```

Approach	When to Use
Cluster	Scaling network applications across CPU cores
child_process	Running external programs or scripts
Worker Threads	CPU-intensive JavaScript computation with shared memory

Chapter 11: Node.js Best Practices

11.1 Project Structure

```
my-app/
├── src/
│   ├── config/           # Configuration files
│   ├── controllers/      # Request handlers
│   ├── models/           # Database models
│   ├── routes/           # Route definitions
│   ├── middlewares/      # Custom middleware
│   ├── services/         # Business logic
│   ├── utils/            # Helper functions
│   └── app.js            # App entry point
├── tests/
├── .env                  # Environment variables (not in git)
├── .gitignore
├── package.json
└── README.md
```

11.2 Coding Best Practices

- Use **async/await** instead of nested callbacks for cleaner code.
- **Always handle errors** at every async boundary; never swallow them silently.
- Use **environment variables** for configuration; never hardcode secrets.
- **Validate input** from clients before processing (use Joi, Yup, or express-validator).
- Use a **linter** (ESLint) and code formatter (Prettier) for consistency.
- **Write tests** with Jest, Mocha, or another framework.
- Use a **process manager** like PM2 in production to handle restarts and clustering.
- **Log structurally** using Winston or Pino instead of console.log.
- **Set security headers** using Helmet, and protect against common attacks (CSRF, XSS).
- **Update dependencies regularly** and audit for vulnerabilities with npm audit.
- Use **TypeScript** for larger projects to catch errors at compile time.
- **Document your APIs** using OpenAPI/Swagger.

11.3 Performance Tips

- Use streams for large file operations to avoid loading everything into memory.
- Cache frequently accessed data with Redis or in-memory caches.
- Use connection pooling for databases.
- Compress responses with the compression middleware.

- Use `Promise.all()` for independent async operations.
- Avoid blocking the event loop with synchronous operations.
- Offload CPU-intensive tasks to worker threads or external services.
- Use HTTP/2 or HTTP/3 for better network performance.

PART II

Express.js

PART II

Express.js

Chapter 12: Introduction to Express.js

12.1 What is Express.js?

Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for building web and mobile applications and APIs. Created by TJ Holowaychuk in 2010, Express has become the de facto standard server framework for Node.js. It is described as the "E" in the popular MEAN, MERN, and MEVN stacks.

12.2 Why Express.js?

- **Minimalist:** Provides just enough structure without being opinionated.
- **Middleware-based:** Easy to add functionality via composable middleware.
- **Routing:** Robust routing system based on HTTP methods and URLs.
- **Templating:** Supports many templating engines (Pug, EJS, Handlebars).
- **Performance:** Thin layer over Node.js with minimal overhead.
- **Large Ecosystem:** Massive community and thousands of compatible packages.
- **Easy to Learn:** Simple, intuitive API that's easy to pick up.

12.3 Setting Up an Express Application

```
# Initialize project and install Express
mkdir my-express-app && cd my-express-app
npm init -y
npm install express
npm install --save-dev nodemon
```

Update `package.json` scripts:

```
"scripts": {
  "start": "node index.js",
  "dev": "nodemon index.js"
}
```

12.4 Hello World in Express


```
// index.js
const express = require('express');
const app = express();
const PORT = process.env.PORT || 3000;

// Built-in middleware to parse JSON bodies
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Define a route
app.get('/', (req, res) => {
  res.send('Hello, World!');
});

// Start the server
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Run with **npm run dev** and visit **http://localhost:3000**.

12.5 The Application Object

The **app** object represents your Express application. It provides methods for routing HTTP requests, configuring middleware, rendering HTML views, registering template engines, and modifying application settings.

Method	Description
app.use()	Mount middleware functions
app.get(), post(), put(), delete()	Route HTTP requests by method
app.all()	Match any HTTP method for a path
app.listen()	Bind and listen for connections on a port
app.set(name, value)	Set application setting
app.get(name)	Get application setting
app.locals	Variables available to all templates
app.engine()	Register a template engine

Chapter 13: Routing in Express

13.1 Basic Routing

Routing refers to how an application's endpoints (URIs) respond to client requests. Each route can have one or more handler functions, which are executed when the route is matched.

```
// Basic route structure: app.METHOD(PATH, HANDLER)

app.get('/users', (req, res) => {
  res.json({ users: ['Alice', 'Bob'] });
});

app.post('/users', (req, res) => {
  res.status(201).json({ created: req.body });
});

app.put('/users/:id', (req, res) => {
  res.json({ updated: req.params.id });
});

app.delete('/users/:id', (req, res) => {
  res.status(204).send();
});

// Match all HTTP methods for a path
app.all('/secret', (req, res) => {
  res.send('Accessed the secret section');
});
```

13.2 Route Parameters

```
// Named route parameters
app.get('/users/:userId', (req, res) => {
  res.json({ userId: req.params.userId });
});

// Multiple parameters
app.get('/users/:userId/posts/:postId', (req, res) => {
  const { userId, postId } = req.params;
  res.json({ userId, postId });
});

// Optional parameter (with ?)
app.get('/users/:userId/:action?', (req, res) => {
  const { userId, action = 'view' } = req.params;
  res.json({ userId, action });
});

// Regex constraint on parameter
app.get('/posts/:id(\\d+)', (req, res) => {
  res.json({ id: parseInt(req.params.id) });
});
```

13.3 Query Strings

```
// URL: /search?q=express&page=2&limit=10
app.get('/search', (req, res) => {
  const { q, page = 1, limit = 10 } = req.query;
  res.json({
    query: q,
    page: parseInt(page),
    limit: parseInt(limit)
  });
});
```

13.4 Route Handlers and Chaining

```
// Multiple handlers (next() moves to the next handler)
app.get('/example',
  (req, res, next) => {
    console.log('First handler');
    next();
  },
  (req, res) => {
    console.log('Second handler');
    res.send('Done');
  }
);

// Chaining methods on the same path
app.route('/users')
  .get((req, res) => res.json({ list: [] })))
  .post((req, res) => res.status(201).json({ created: true })))
  .put((req, res) => res.json({ updated: true }));
```

13.5 express.Router()

express.Router() creates a modular, mountable route handler. A Router instance is a complete middleware and routing system; for this reason, it's often referred to as a "mini-app."

```
// routes/users.js
const express = require('express');
const router = express.Router();

// Middleware that runs for every route in this router
router.use((req, res, next) => {
  console.log('User route hit:', req.method, req.url);
  next();
});

router.get('/', (req, res) => res.json({ users: [] }));
router.post('/', (req, res) => res.status(201).json({ created: true }));
router.get('/:id', (req, res) => res.json({ id: req.params.id }));

module.exports = router;

// index.js – mount the router
const usersRouter = require('./routes/users');
app.use('/api/users', usersRouter);
// Now: GET /api/users, POST /api/users, GET /api/users/:id
```

Chapter 14: Middleware

14.1 What is Middleware?

Middleware functions are functions that have access to the request object (req), the response object (res), and the next middleware function (next) in the application's request-response cycle. Middleware can execute code, modify req and res, end the cycle, or call the next middleware. This is one of the most important concepts in Express and a top interview topic.

```
// Anatomy of a middleware function
function myMiddleware(req, res, next) {
  // 1. Execute any code
  console.log(`${req.method} ${req.url}`);

  // 2. Make changes to req/res
  req.timestamp = Date.now();

  // 3. Either end the cycle or call next()
  next(); // Pass control to the next middleware
}

app.use(myMiddleware);
```

14.2 Types of Middleware

Type	Description
Application-level	Bound to the app object using app.use() and app.METHOD()
Router-level	Bound to express.Router() instances
Error-handling	Takes four arguments: (err, req, res, next)
Built-in	express.json(), express.urlencoded(), express.static()
Third-party	From npm: morgan, helmet, cors, compression, etc.

14.3 Application-Level Middleware

```
// Runs for every request
app.use((req, res, next) => {
  console.log('Time:', Date.now());
  next();
});

// Runs only for a specific path
app.use('/admin', (req, res, next) => {
  console.log('Admin route accessed');
  next();
});

// Runs only for a specific path + method
app.get('/users',
  (req, res, next) => {
    console.log('GET /users middleware');
    next();
  },
  (req, res) => res.json({ users: [] })
);
```

14.4 Built-in Middleware

```
// Parse JSON request bodies
app.use(express.json({ limit: '10mb' }));

// Parse URL-encoded request bodies (HTML forms)
app.use(express.urlencoded({ extended: true }));

// Serve static files from the 'public' directory
app.use(express.static('public'));

// You can also mount static files at a virtual path
app.use('/static', express.static('public'));
```

14.5 Common Third-Party Middleware

Package	Purpose
morgan	HTTP request logger
helmet	Sets various HTTP headers for security
cors	Enables Cross-Origin Resource Sharing
compression	Compresses response bodies (gzip)
cookie-parser	Parse Cookie header into req.cookies
express-session	Session middleware
multer	Handle multipart/form-data (file uploads)
express-rate-limit	Basic rate limiting
express-validator	Request data validation
passport	Authentication middleware

```
const express = require('express');
const morgan = require('morgan');
const helmet = require('helmet');
const cors = require('cors');
const compression = require('compression');

const app = express();

app.use(helmet()); // Security headers
app.use(cors()); // Enable CORS
app.use(compression()); // Gzip responses
app.use(morgan('combined')); // Request logging
app.use(express.json()); // JSON body parsing
app.use(express.urlencoded({ extended: true }));
```

14.6 Custom Middleware Examples

```
// Request logger middleware
function requestLogger(req, res, next) {
  const start = Date.now();
  res.on('finish', () => {
    const duration = Date.now() - start;
    console.log(`${req.method} ${req.url} ${res.statusCode} - ${duration}ms`);
  });
  next();
}

// Authentication middleware
function authenticate(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) {
    return res.status(401).json({ error: 'No token provided' });
  }
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (err) {
    res.status(401).json({ error: 'Invalid token' });
  }
}

// Authorization middleware (role-based)
function authorize(...allowedRoles) {
  return (req, res, next) => {
    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Forbidden' });
    }
    next();
  };
}

// Usage
app.use(requestLogger);
app.get('/admin', authenticate, authorize('admin'), (req, res) => {
  res.json({ secret: 'data' });
});
```

14.7 Error-Handling Middleware

Error-handling middleware is identified by having **four** arguments: `(err, req, res, next)`. It must be defined **after** all other middleware and routes.


```
// Application code throws or passes errors to next()
app.get('/api/users/:id', async (req, res, next) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) {
      const error = new Error('User not found');
      error.statusCode = 404;
      throw error;
    }
    res.json(user);
  } catch (err) {
    next(err); // Forward to error middleware
  }
});

// 404 handler (for unmatched routes) – placed after all routes
app.use((req, res, next) => {
  res.status(404).json({ error: 'Route not found' });
});

// Centralized error handler
app.use((err, req, res, next) => {
  console.error(err.stack);
  const statusCode = err.statusCode || 500;
  res.status(statusCode).json({
    error: {
      message: err.message || 'Internal Server Error',
      ...(process.env.NODE_ENV === 'development' && { stack: err.stack })
    }
  });
});
```

IMPORTANT: The order of middleware matters! Express executes middleware in the order they are registered. Always place error-handling middleware *last*, and 404 handlers *after* all valid routes.

Chapter 15: Request and Response Objects

15.1 Request Object (req)

Property / Method	Description
req.params	Route parameters (e.g., :id)
req.query	Parsed query string parameters
req.body	Parsed request body (requires body-parsing middleware)
req.headers	Request headers
req.cookies	Parsed cookies (requires cookie-parser)
req.method	HTTP method (GET, POST, etc.)
req.url	Request URL
req.path	Path part of the URL
req.hostname	Host name from Host header
req.ip	Remote IP address
req.protocol	'http' or 'https'
req.secure	true if HTTPS
req.xhr	true if XMLHttpRequest
req.get(field)	Get a request header by name

15.2 Response Object (res)

Method	Description
res.send(body)	Send a response (string, buffer, object, array)
res.json(obj)	Send a JSON response
res.status(code)	Set the HTTP status code (chainable)
res.sendStatus(code)	Set status and send the status text
res.redirect(url)	Redirect to another URL
res.render(view, data)	Render a view template
res.set(field, value)	Set a response header
res.cookie(name, value)	Set a cookie
res.clearCookie(name)	Clear a cookie
res.download(path)	Send a file as a download
res.sendFile(path)	Send a file as the response
res.end()	End the response without data

```
app.get('/example', (req, res) => {
  // Set status and headers (chainable)
  res.status(200)
    .set('X-Custom-Header', 'value')
    .cookie('session_id', 'abc123', {
      httpOnly: true,
      secure: true,
      maxAge: 24 * 60 * 60 * 1000 // 1 day
    })
    .json({ message: 'Success' });
});

// Redirect
app.get('/old', (req, res) => res.redirect(301, '/new'));

// Send a file
app.get('/download', (req, res) => {
  res.download('/path/to/file.pdf', 'custom-name.pdf');
});
```

15.3 Common HTTP Status Codes

Code	Meaning	Use Case
200	OK	Successful GET, PUT, PATCH
201	Created	Successful POST that creates a resource
204	No Content	Successful DELETE or update with no body
301	Moved Permanently	Permanent redirect
302	Found	Temporary redirect
304	Not Modified	Resource hasn't changed (caching)
400	Bad Request	Invalid request data or syntax
401	Unauthorized	Authentication required or invalid
403	Forbidden	Authenticated but no permission
404	Not Found	Resource doesn't exist
409	Conflict	Resource conflict (e.g., duplicate)
422	Unprocessable Entity	Validation failed
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Server error
503	Service Unavailable	Server overloaded or down for maintenance

Chapter 16: Building REST APIs

16.1 REST Principles

REST (Representational State Transfer) is an architectural style for designing networked applications. RESTful APIs use HTTP requests to perform CRUD operations on resources.

- **Stateless:** Each request contains all information needed; server stores no client context.
- **Client-Server:** Clear separation between client and server.
- **Cacheable:** Responses define themselves as cacheable or non-cacheable.
- **Uniform Interface:** Resources identified by URIs; standard methods (GET, POST, PUT, DELETE).
- **Layered System:** Client cannot tell whether it's connected directly to the end server.
- **Resource-based:** Everything is a resource accessed via URIs.

16.2 REST API Design

HTTP Method	URL Pattern	Description
GET	/api/users	List all users
GET	/api/users/:id	Get a single user
POST	/api/users	Create a new user
PUT	/api/users/:id	Replace an existing user
PATCH	/api/users/:id	Partially update a user
DELETE	/api/users/:id	Delete a user
GET	/api/users/:id/posts	Get posts by a user (nested resource)

16.3 Complete CRUD Example

```
// routes/products.js
const express = require('express');
const router = express.Router();
const Product = require('../models/Product');

// GET /api/products?page=1&limit=10
router.get('/', async (req, res, next) => {
  try {
    const { page = 1, limit = 10, sort = '-createdAt' } = req.query;
    const products = await Product.find()
      .sort(sort)
      .skip((page - 1) * limit)
      .limit(parseInt(limit));
    const total = await Product.countDocuments();
    res.json({
      data: products,
      page: parseInt(page),
      limit: parseInt(limit),
      total,
      pages: Math.ceil(total / limit)
    });
  } catch (err) { next(err); }
});

// GET /api/products/:id
router.get('/:id', async (req, res, next) => {
  try {
    const product = await Product.findById(req.params.id);
    if (!product) return res.status(404).json({ error: 'Not found' });
    res.json(product);
  } catch (err) { next(err); }
});
```

```
    } catch (err) { next(err); }
  });

// POST /api/products
router.post('/', async (req, res, next) => {
  try {
    const product = await Product.create(req.body);
    res.status(201).json(product);
  } catch (err) { next(err); }
});

// PATCH /api/products/:id
router.patch('/:id', async (req, res, next) => {
  try {
    const product = await Product.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );
    if (!product) return res.status(404).json({ error: 'Not found' });
    res.json(product);
  } catch (err) { next(err); }
});

// DELETE /api/products/:id
router.delete('/:id', async (req, res, next) => {
  try {
    const product = await Product.findByIdAndDelete(req.params.id);
    if (!product) return res.status(404).json({ error: 'Not found' });
    res.status(204).send();
  } catch (err) { next(err); }
});

module.exports = router;
```

16.4 API Versioning

When you make breaking changes to your API, versioning lets old clients continue working. Common strategies:

- **URL versioning:** /api/v1/users, /api/v2/users (most common)
- **Header versioning:** Accept: application/vnd.myapp.v2+json
- **Query parameter:** /api/users?version=2

```
app.use('/api/v1/users', require('./routes/v1/users'));
app.use('/api/v2/users', require('./routes/v2/users'));
```

Chapter 17: Authentication and Authorization

17.1 Authentication vs Authorization

Authentication	Authorization
Who are you?	What are you allowed to do?
Verifies identity	Verifies permissions
Login forms, tokens, biometrics	Role checks, access control lists
Usually happens first	Happens after authentication

17.2 Password Hashing with bcrypt

Never store passwords in plain text. Use bcrypt to hash passwords with a salt — even if your database is compromised, the passwords remain secure.

```
npm install bcrypt
```

```
const bcrypt = require('bcrypt');
const SALT_ROUNDS = 12;

// Hash a password during registration
async function register(email, password) {
  const hash = await bcrypt.hash(password, SALT_ROUNDS);
  await User.create({ email, password: hash });
}

// Verify a password during login
async function login(email, password) {
  const user = await User.findOne({ email });
  if (!user) throw new Error('Invalid credentials');
  const valid = await bcrypt.compare(password, user.password);
  if (!valid) throw new Error('Invalid credentials');
  return user;
}
```


17.3 JSON Web Tokens (JWT)

JWT is a compact, URL-safe means of representing claims between two parties. A JWT consists of three parts separated by dots: header, payload, and signature. JWTs are stateless — the server doesn't need to store session data, making them ideal for distributed systems.

```
npm install jsonwebtoken
```

```
const jwt = require('jsonwebtoken');
const JWT_SECRET = process.env.JWT_SECRET;

// Create a token at login
function login(req, res) {
  // ... verify credentials ...
  const token = jwt.sign(
    { userId: user._id, role: user.role },
    JWT_SECRET,
    { expiresIn: '1h' }
  );
  res.json({ token });
}

// Verify token in middleware
function authenticate(req, res, next) {
  const header = req.headers.authorization;
  if (!header || !header.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'No token provided' });
  }
  const token = header.split(' ')[1];
  try {
    req.user = jwt.verify(token, JWT_SECRET);
    next();
  } catch (err) {
    if (err.name === 'TokenExpiredError') {
      return res.status(401).json({ error: 'Token expired' });
    }
    return res.status(401).json({ error: 'Invalid token' });
  }
}

// Use the middleware on protected routes
app.get('/api/profile', authenticate, (req, res) => {
  res.json({ userId: req.user.userId });
});
```

17.4 Refresh Tokens

Access tokens should be short-lived (e.g., 15 minutes). Use refresh tokens (longer-lived, e.g., 7 days) to obtain new access tokens without requiring the user to log in again.

```
// Issue both tokens at login
const accessToken = jwt.sign({ userId }, ACCESS_SECRET, { expiresIn: '15m' });
const refreshToken = jwt.sign({ userId }, REFRESH_SECRET, { expiresIn: '7d' });
// Store refresh token in DB (so it can be revoked)
await RefreshToken.create({ userId, token: refreshToken });

// Endpoint to refresh
app.post('/api/refresh', async (req, res) => {
  const { refreshToken } = req.body;
  const stored = await RefreshToken.findOne({ token: refreshToken });
  if (!stored) return res.status(401).json({ error: 'Invalid token' });
  try {
    const decoded = jwt.verify(refreshToken, REFRESH_SECRET);
    const newAccess = jwt.sign(
      { userId: decoded.userId },
      ACCESS_SECRET,
      { expiresIn: '15m' }
    );
    res.json({ accessToken: newAccess });
  } catch (err) {
    res.status(401).json({ error: 'Invalid refresh token' });
  }
});
```

17.5 Sessions vs JWTs

Aspect	Sessions	JWT
Storage	Server-side	Client-side (token contains data)
Scalability	Needs shared store (Redis)	Stateless, easy to scale
Revocation	Easy — delete from store	Hard — needs blacklist
Size	Small cookie (session ID)	Larger token (signed payload)
CSRF risk	Vulnerable (need CSRF tokens)	Reduced if not stored in cookies
Best for	Traditional web apps	APIs, mobile apps, SPAs

17.6 Role-Based Access Control (RBAC)

```
// Authorization middleware factory
function authorize(...allowedRoles) {
  return (req, res, next) => {
    if (!req.user) {
      return res.status(401).json({ error: 'Not authenticated' });
    }
    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Forbidden' });
    }
    next();
  };
}

// Apply to routes
app.get('/admin/users', authenticate, authorize('admin'), getAllUsers);
app.delete('/admin/users/:id', authenticate, authorize('admin'), deleteUser);
app.get('/api/orders', authenticate, authorize('user', 'admin'), getOrders);
```

Chapter 18: Input Validation

18.1 Why Validate?

Never trust client input. Validation protects your application from malformed data, security vulnerabilities (SQL injection, NoSQL injection, XSS), and unexpected errors. Validate input as early as possible — preferably in middleware before reaching business logic.

18.2 express-validator

```
npm install express-validator
```

```
const { body, param, query, validationResult } = require('express-validator');

// Validation middleware
const validateUser = [
  body('email').isEmail().normalizeEmail()
    .withMessage('Invalid email'),
  body('password').isLength({ min: 8 })
    .withMessage('Password must be at least 8 characters')
    .matches(/[A-Z]/)
    .withMessage('Must contain an uppercase letter')
    .matches(/[0-9]/)
    .withMessage('Must contain a number'),
  body('age').optional().isInt({ min: 18, max: 120 })
    .withMessage('Age must be between 18 and 120'),
  body('username').isAlphanumeric().isLength({ min: 3, max: 20 })
    .withMessage('Username must be 3-20 alphanumeric characters'),
];

// Handler that checks for validation errors
function checkValidation(req, res, next) {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(422).json({ errors: errors.array() });
  }
  next();
}

app.post('/api/register', validateUser, checkValidation, (req, res) => {
  // Body data is validated and sanitized
  res.status(201).json({ created: req.body });
});
```

18.3 Joi Validation

Joi is another popular choice with a schema-first approach.

```
npm install joi
```

```
const Joi = require('joi');

const userSchema = Joi.object({
  email: Joi.string().email().required(),
  password: Joi.string().min(8).pattern(/[A-Z]/).pattern(/[0-9]/).required(),
  age: Joi.number().integer().min(18).max(120).optional(),
  role: Joi.string().valid('user', 'admin').default('user'),
  preferences: Joi.object({
    newsletter: Joi.boolean().default(false),
    theme: Joi.string().valid('light', 'dark')
  }).optional()
});

// Generic validator middleware factory
function validate(schema) {
  return (req, res, next) => {
    const { error, value } = schema.validate(req.body, {
      abortEarly: false,
      stripUnknown: true
    });
    if (error) {
      return res.status(422).json({
        errors: error.details.map(d => d.message)
      });
    }
    req.body = value; // Use sanitized value
    next();
  };
}

app.post('/api/users', validate(userSchema), createUser);
```

Chapter 19: File Uploads with Multer

19.1 Setting Up Multer

```
npm install multer
```

```
const express = require('express');
const multer = require('multer');
const path = require('path');

// Disk storage configuration
const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, 'uploads/'),
  filename: (req, file, cb) => {
    const unique = Date.now() + '-' + Math.round(Math.random() * 1E9);
    cb(null, unique + path.extname(file.originalname));
  }
});

// File filter
function fileFilter(req, file, cb) {
  const allowed = /jpeg|jpg|png|gif|pdf/;
  const extOk = allowed.test(path.extname(file.originalname).toLowerCase());
  const mimeOk = allowed.test(file.mimetype);
  if (extOk && mimeOk) return cb(null, true);
  cb(new Error('Only images and PDFs are allowed'));
}

const upload = multer({
  storage,
  fileFilter,
  limits: {
    fileSize: 5 * 1024 * 1024, // 5 MB
    files: 10
  }
});
```

19.2 Single and Multiple File Uploads

```
// Single file upload (field name 'avatar')
app.post('/upload/avatar', upload.single('avatar'), (req, res) => {
  res.json({
    filename: req.file.filename,
    path: req.file.path,
    size: req.file.size
  });
});

// Multiple files in one field (up to 5)
app.post('/upload/photos', upload.array('photos', 5), (req, res) => {
  res.json({ files: req.files.map(f => f.filename) });
});

// Multiple fields with multiple files each
app.post('/upload/mixed', upload.fields([
  { name: 'avatar', maxCount: 1 },
  { name: 'gallery', maxCount: 8 }
]), (req, res) => {
  res.json({
    avatar: req.files.avatar?.[0]?.filename,
    gallery: req.files.gallery?.map(f => f.filename)
  });
});
```

Chapter 20: Express Security Best Practices

20.1 Common Security Threats

Threat	Description
XSS (Cross-Site Scripting)	Injecting malicious scripts into web pages
CSRF (Cross-Site Request Forgery)	Tricking users into submitting requests they didn't intend
SQL/NoSQL Injection	Injecting code into database queries
Brute Force Attacks	Automated login attempts with many passwords
DDoS	Overwhelming server with traffic
Sensitive Data Exposure	Leaking secrets in error messages, logs, or responses
Insecure Dependencies	Using libraries with known vulnerabilities

20.2 Helmet — Security Headers

```
const helmet = require('helmet');
app.use(helmet());

// Helmet sets the following headers by default:
// • Content-Security-Policy
// • Cross-Origin-Embedder-Policy
// • Cross-Origin-Opener-Policy
// • Cross-Origin-Resource-Policy
// • Origin-Agent-Cluster
// • Referrer-Policy
// • Strict-Transport-Security (HSTS)
// • X-Content-Type-Options: nosniff
// • X-DNS-Prefetch-Control
// • X-Download-Options
// • X-Frame-Options
// • X-Permitted-Cross-Domain-Policies
// • X-XSS-Protection
```

20.3 CORS Configuration


```
const cors = require('cors');

// Simple – allow all origins (development only!)
app.use(cors());

// Production – restrict to specific origins
const corsOptions = {
  origin: ['https://example.com', 'https://app.example.com'],
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  credentials: true, // Allow cookies
  maxAge: 86400 // Cache preflight for 24 hours
};
app.use(cors(corsOptions));

// Dynamic origin function
app.use(cors({
  origin: (origin, callback) => {
    const allowed = ['https://example.com', 'https://app.example.com'];
    if (!origin || allowed.includes(origin)) callback(null, true);
    else callback(new Error('Not allowed by CORS'));
  }
}));
```

20.4 Rate Limiting

```
const rateLimit = require('express-rate-limit');

// Apply to all requests
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // 100 requests per IP
  message: 'Too many requests',
  standardHeaders: true, // RateLimit headers
  legacyHeaders: false // Disable X-RateLimit-* headers
});
app.use(limiter);

// Stricter for sensitive routes
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: 'Too many login attempts; please try again in 15 minutes'
});
app.post('/api/login', loginLimiter, loginHandler);
```

20.5 Other Security Tips

- Always use HTTPS in production (TLS certificates from Let's Encrypt).
- Use environment variables for all secrets; never commit .env files.
- Sanitize user input with libraries like express-mongo-sanitize and xss-clean.

- Use parameterized queries / ORM methods to prevent injection.
- Set secure, httpOnly, and sameSite flags on cookies.
- Disable the X-Powered-By header (Helmet does this automatically).
- Keep dependencies updated and run npm audit regularly.
- Log security events but avoid logging sensitive data (passwords, tokens).
- Implement proper CORS policies — don't use wildcard origins in production.
- Validate file uploads — check size, type, and use virus scanning if needed.

Chapter 21: Testing Express Applications

21.1 Testing Setup

```
npm install --save-dev jest supertest
```

Add to package.json:

```
"scripts": {  
  "test": "jest",  
  "test:watch": "jest --watch",  
  "test:coverage": "jest --coverage"  
}
```

21.2 Unit Tests with Jest

```
// utils/math.js  
exports.add = (a, b) => a + b;  
exports.divide = (a, b) => {  
  if (b === 0) throw new Error('Cannot divide by zero');  
  return a / b;  
};  
  
// utils/math.test.js  
const { add, divide } = require('./math');  
  
describe('Math utilities', () => {  
  test('add: sums two numbers', () => {  
    expect(add(2, 3)).toBe(5);  
    expect(add(-1, 1)).toBe(0);  
  });  
  
  test('divide: returns quotient', () => {  
    expect(divide(10, 2)).toBe(5);  
  });  
  
  test('divide: throws on division by zero', () => {  
    expect(() => divide(10, 0)).toThrow('Cannot divide by zero');  
  });  
});
```

21.3 Integration Tests with Supertest

```
// app.test.js
const request = require('supertest');
const app = require('./app');

describe('GET /api/users', () => {
  it('responds with JSON array', async () => {
    const res = await request(app)
      .get('/api/users')
      .expect('Content-Type', /json/)
      .expect(200);
    expect(Array.isArray(res.body)).toBe(true);
  });
});

describe('POST /api/users', () => {
  it('creates a new user', async () => {
    const userData = {
      email: 'test@example.com',
      password: 'Secret123'
    };
    const res = await request(app)
      .post('/api/users')
      .send(userData)
      .expect(201);
    expect(res.body.email).toBe(userData.email);
    expect(res.body).not.toHaveProperty('password');
  });

  it('rejects invalid email', async () => {
    await request(app)
      .post('/api/users')
      .send({ email: 'not-an-email', password: 'Secret123' })
      .expect(422);
  });
});
```

PART III

MongoDB

Chapter 22: Introduction to MongoDB

22.1 What is MongoDB?

MongoDB is a popular, open-source, document-oriented NoSQL database developed by MongoDB Inc. and first released in 2009. It stores data in flexible, JSON-like documents called BSON (Binary JSON). MongoDB is designed for scalability, high performance, and high availability, making it ideal for modern web applications.

22.2 SQL vs NoSQL

Aspect	SQL (Relational)	NoSQL (MongoDB)
Data Model	Tables with rows and columns	Collections of documents
Schema	Fixed, predefined	Flexible, dynamic
Relationships	Joins via foreign keys	Embedded docs or references
Scalability	Vertical (bigger server)	Horizontal (more servers / sharding)
Query Language	SQL	MongoDB Query Language (MQL)
ACID	Full ACID compliance	ACID at document level; multi-doc since 4.0
Best For	Complex transactions, structured data	Big data, rapid iteration, hierarchies
Examples	PostgreSQL, MySQL, Oracle	MongoDB, Couchbase, DynamoDB

22.3 Key MongoDB Concepts

MongoDB Term	SQL Equivalent	Description
Database	Database	Container for collections
Collection	Table	Group of documents (schema-less)
Document	Row / Record	JSON-like object (BSON)
Field	Column	Key-value pair in a document
_id	Primary Key	Unique identifier (auto-generated ObjectId)
Embedded Doc	Joined Tables	A document inside another document
Reference	Foreign Key	_id pointing to another document
Index	Index	Performance optimization for queries

22.4 BSON Documents

BSON (Binary JSON) is a binary-encoded serialization of JSON-like documents. It supports additional data types not in JSON like Date, ObjectId, Binary, and Decimal128.

```
// Example MongoDB document
{
  "_id": ObjectId("507f1f77bcf86cd799439011"),
  "name": "John Doe",
  "email": "john@example.com",
  "age": 30,
  "isActive": true,
  "joinedAt": ISODate("2023-06-15T10:30:00Z"),
  "address": {
    "street": "123 Main St",
    "city": "Boston",
    "zip": "02101"
  },
  "hobbies": ["reading", "coding", "hiking"],
  "scores": [
    { "subject": "Math", "score": 95 },
    { "subject": "English", "score": 88 }
  ]
}
```

22.5 Installation and Tools

- **MongoDB Community Server:** The local installation for development.
- **MongoDB Atlas:** Fully-managed cloud database service (free tier available).
- **MongoDB Compass:** Official GUI for visualizing and managing data.
- **MongoDB Shell (mongosh):** Modern command-line interface.

- **MongoDB Database Tools:** mongodump, mongorestore, mongoimport, mongoexport.

Chapter 23: CRUD Operations

23.1 Connecting to MongoDB

```
# Connect with mongosh
mongosh "mongodb://localhost:27017"
mongosh "mongodb+srv://user:pass@cluster.mongodb.net/myDB"

# Within the shell
show dbs                // List databases
use shopDB              // Switch / create database
show collections        // List collections
db.users.countDocuments() // Count documents
```

23.2 Create Operations

```
// Insert a single document
db.users.insertOne({
  name: 'Alice',
  email: 'alice@example.com',
  age: 28
});
// Returns: { acknowledged: true, insertedId: ObjectId(...) }

// Insert multiple documents
db.users.insertMany([
  { name: 'Bob', email: 'bob@example.com', age: 32 },
  { name: 'Carol', email: 'carol@example.com', age: 24 },
  { name: 'Dave', email: 'dave@example.com', age: 45 }
]);

// Insert with ordered: false continues on errors
db.users.insertMany([...], { ordered: false });
```

23.3 Read Operations

```
// Find all documents in a collection
db.users.find();

// Find with criteria
db.users.find({ age: 28 });
db.users.find({ name: 'Alice', age: 28 }); // AND

// Find one (returns first match)
db.users.findOne({ email: 'alice@example.com' });

// Projection – select specific fields
db.users.find({}, { name: 1, email: 1, _id: 0 });
// 1 = include, 0 = exclude, _id is included by default

// Comparison operators
db.users.find({ age: { $gt: 25 } }); // age > 25
db.users.find({ age: { $gte: 25, $lte: 40 } }); // 25 <= age <= 40
db.users.find({ name: { $in: ['Alice', 'Bob'] } });
db.users.find({ name: { $ne: 'Alice' } }); // not equal

// Logical operators
db.users.find({ $or: [{ age: { $lt: 25 } }, { age: { $gt: 40 } }] });
db.users.find({ $and: [{ age: { $gte: 25 } }, { age: { $lte: 40 } }] });
db.users.find({ age: { $not: { $gte: 30 } } });

// Existence check
db.users.find({ phone: { $exists: true } });
db.users.find({ phone: { $exists: false } });

// Regex (pattern matching)
db.users.find({ name: /^A/ }); // starts with A
db.users.find({ email: /@gmail.com$/ }); // ends with @gmail.com
```

23.4 Sorting, Limiting, and Skipping

```
// Sort: 1 = ascending, -1 = descending
db.users.find().sort({ age: -1 });           // oldest first
db.users.find().sort({ name: 1, age: -1 }); // multi-field sort

// Limit number of results
db.users.find().limit(10);

// Skip results (for pagination)
db.users.find().skip(20).limit(10); // page 3 with size 10

// Combine for pagination
const page = 2, pageSize = 10;
db.users.find()
  .sort({ createdAt: -1 })
  .skip((page - 1) * pageSize)
  .limit(pageSize);

// Count results
db.users.countDocuments({ age: { $gte: 18 } });
db.users.estimatedDocumentCount(); // fast, uses metadata
```

23.5 Update Operations

```
// Update one document
db.users.updateOne(
  { email: 'alice@example.com' }, // filter
  { $set: { age: 29 } }           // update
);

// Update many documents
db.users.updateMany(
  { age: { $lt: 18 } },
  { $set: { isMinor: true } }
);

// Replace a document entirely (keeps _id)
db.users.replaceOne(
  { email: 'alice@example.com' },
  { name: 'Alice Smith', email: 'alice@example.com', age: 29 }
);

// Upsert – update if exists, insert if not
db.users.updateOne(
  { email: 'new@example.com' },
  { $set: { name: 'New User', age: 25 } },
  { upsert: true }
);

// findOneAndUpdate – returns the document
db.users.findOneAndUpdate(
  { _id: userId },
  { $inc: { loginCount: 1 } },
  { returnDocument: 'after' } // 'before' or 'after'
);
```

Update Operators

Operator	Description	Example
\$set	Set field value	{ \$set: { name: 'Alice' } }
\$unset	Remove field	{ \$unset: { phone: '' } }
\$inc	Increment numeric field	{ \$inc: { age: 1, views: 10 } }
\$mul	Multiply numeric field	{ \$mul: { price: 1.1 } }
\$rename	Rename field	{ \$rename: { 'old': 'new' } }
\$min	Update if new < current	{ \$min: { lowScore: 50 } }
\$max	Update if new > current	{ \$max: { highScore: 100 } }
\$currentDate	Set to current date	{ \$currentDate: { updatedAt: true } }
\$push	Append to array	{ \$push: { tags: 'new' } }
\$addToSet	Add to array if not exists	{ \$addToSet: { tags: 'unique' } }
\$pull	Remove items from array	{ \$pull: { tags: 'old' } }
\$pop	Remove first/last array item	{ \$pop: { tags: 1 } } // -1 for first
\$each	Used with \$push for multiple	{ \$push: { tags: { \$each: ['a','b'] } } }

23.6 Delete Operations

```
// Delete one document
db.users.deleteOne({ email: 'alice@example.com' });

// Delete many documents
db.users.deleteMany({ isActive: false });

// Delete all documents in collection (collection still exists)
db.users.deleteMany({});

// Drop entire collection (faster, also removes indexes)
db.users.drop();

// Drop entire database
db.dropDatabase();

// findOneAndDelete – returns the deleted document
const deleted = db.users.findOneAndDelete({ _id: userId });
```

Chapter 24: Advanced Queries

24.1 Query Operators Reference

Operator	Description
\$eq	Equal to value
\$ne	Not equal to value
\$gt / \$gte	Greater than / Greater than or equal
\$lt / \$lte	Less than / Less than or equal
\$in	In an array of values
\$nin	Not in an array of values
\$and	Logical AND
\$or	Logical OR
\$nor	Logical NOR
\$not	Logical NOT
\$exists	Field exists or not
\$type	Match BSON type
\$regex	Match regular expression
\$mod	Modulo: matches docs where field % divisor == remainder
\$text	Text search (requires text index)
\$where	Match using JavaScript (slow, avoid if possible)
\$expr	Use aggregation expressions in queries

24.2 Array Query Operators

```
// Document: { name: 'Alice', tags: ['admin', 'user'], scores: [85, 92, 78] }

// Match any element in array
db.users.find({ tags: 'admin' });           // contains 'admin'
db.users.find({ scores: { $gt: 90 } });      // any score > 90

// $all – all elements must match
db.users.find({ tags: { $all: ['admin', 'user'] } });

// $elemMatch – at least one element matches ALL conditions
db.users.find({
  scores: { $elemMatch: { $gte: 80, $lt: 90 } }
});

// $size – array length
db.users.find({ tags: { $size: 2 } });

// Query by array index (zero-based)
db.users.find({ 'scores.0': { $gt: 90 } }); // first score > 90
```

24.3 Querying Embedded Documents

```
// Document: { name: 'Alice', address: { city: 'Boston', zip: '02101' } }

// Exact match of embedded document (order matters!)
db.users.find({ address: { city: 'Boston', zip: '02101' } });

// Dot notation for nested fields (preferred)
db.users.find({ 'address.city': 'Boston' });
db.users.find({ 'address.city': 'Boston', 'address.zip': '02101' });

// Updating nested fields
db.users.updateOne(
  { _id: userId },
  { $set: { 'address.city': 'New York' } }
);
```

24.4 Cursor Methods

```
// find() returns a cursor
const cursor = db.users.find({ age: { $gt: 25 } });

// Cursor methods can be chained
cursor.sort({ age: -1 }).limit(10).skip(20).toArray();

// Iterate over cursor
cursor.forEach(doc => printjson(doc));

// Convert to array
const docs = cursor.toArray();

// Check if more documents
while (cursor.hasNext()) {
  const doc = cursor.next();
  // process doc
}
```

Chapter 25: Indexes

25.1 Why Indexes?

Indexes support efficient execution of queries in MongoDB. Without indexes, MongoDB must perform a collection scan — examining every document — to select those matching the query. If an appropriate index exists, MongoDB can limit the documents it must inspect, dramatically improving performance. Indexes are essential for production-grade applications.

25.2 Types of Indexes

Index Type	Description
Single Field	Index on a single field, ascending or descending
Compound	Index on multiple fields (order matters)
Multikey	Automatically created on array fields
Text	For text search using \$text operator
Hashed	For hash-based sharding (= queries only, no ranges)
Geospatial	2d and 2dsphere for location-based queries
Wildcard	Match arbitrary fields by pattern
TTL (Time-To-Live)	Auto-delete documents after a period
Unique	Enforces uniqueness on indexed field(s)
Partial	Indexes only docs matching a filter expression
Sparse	Indexes only docs that have the indexed field

25.3 Creating and Managing Indexes

```
// Single field index (ascending)
db.users.createIndex({ email: 1 });

// Compound index – field order matters!
db.users.createIndex({ status: 1, createdAt: -1 });
// Useful for: find({ status: 'active' }).sort({ createdAt: -1 })

// Unique index – duplicates rejected
db.users.createIndex({ email: 1 }, { unique: true });

// Partial index – only indexes documents matching filter
db.users.createIndex(
  { email: 1 },
  { partialFilterExpression: { isActive: true } }
);

// Text index for full-text search
db.articles.createIndex({ title: 'text', content: 'text' });
db.articles.find({ $text: { $search: 'mongodb tutorial' } });

// TTL index – documents auto-deleted after expireAfterSeconds
db.sessions.createIndex(
  { createdAt: 1 },
  { expireAfterSeconds: 3600 } // 1 hour
);

// View all indexes
db.users.getIndexes();

// Drop an index
db.users.dropIndex({ email: 1 });
db.users.dropIndex('email_1'); // by name
```

25.4 Index Performance and explain()

```
// Use explain() to analyze query performance
db.users.find({ age: { $gt: 25 } }).explain('executionStats');

// Key fields in the output:
// - winningPlan.stage: 'COLLSCAN' means full scan (bad!)
// - 'IXSCAN' means index scan (good)
// - executionTimeMillis: how long the query took
// - totalKeysExamined: index entries examined
// - totalDocsExamined: documents examined
// - nReturned: documents returned

// Ideal: totalDocsExamined ≈ nReturned, and stage is IXSCAN
```

25.5 Index Best Practices

- Create indexes that support your queries; use explain() to verify.

- Avoid creating too many indexes — each one slows down writes.
- Place equality fields first in compound indexes, then sort fields, then range fields.
- Use partial indexes when only a subset of documents needs indexing.
- Use covered queries: include all needed fields in the index for fastest reads.
- Drop unused indexes — check with `db.collection.aggregate([{$indexStats: {}}])`.
- Build indexes in the background in production: `{ background: true }` (default for ≥ 4.2).
- Index sizes grow with collection size — monitor RAM usage.

Chapter 26: Aggregation Framework

The aggregation framework processes data records through a pipeline of stages, transforming documents into aggregated results. It's MongoDB's primary tool for analytics, reporting, and complex data transformations — analogous to SQL's GROUP BY, JOIN, and window functions, but much more flexible.

TIP: An aggregation pipeline is an array of stages. Each stage takes documents as input, transforms them, and passes output to the next stage. Order matters and affects performance significantly.

26.1 Pipeline Stages Reference

Stage	Purpose
\$match	Filter documents (like WHERE in SQL). Place early for performance.
\$project	Reshape documents — include, exclude, or compute new fields.
\$group	Group documents by a key and apply accumulators (sum, avg, etc.).
\$sort	Sort documents by one or more fields.
\$limit / \$skip	Limit number of documents / skip documents (pagination).
\$unwind	Deconstruct an array field into separate documents.
\$lookup	Left outer join with another collection.
\$addFields / \$set	Add new fields without removing existing ones.
\$count	Return a count of documents in the pipeline.
\$facet	Run multiple sub-pipelines in parallel on the same input.
\$bucket	Categorize documents into groups based on ranges.
\$out / \$merge	Write pipeline results to a collection.

26.2 Basic Aggregation Example

```
// Sample orders collection
// { _id, customerId, status, total, items: [...], createdAt }

// Total revenue per customer status
db.orders.aggregate([
  { $match: { status: 'completed' } },
  { $group: {
    _id: '$customerId',
    totalSpent: { $sum: '$total' },
    orderCount: { $sum: 1 },
    avgOrder: { $avg: '$total' }
  }},
  { $sort: { totalSpent: -1 } },
  { $limit: 10 }
]);

// Result: top 10 customers by spending
```

26.3 Accumulator Operators (used in \$group)

Operator	Description
\$sum	Sum of numeric values. \$sum: 1 counts documents.
\$avg	Average of numeric values.
\$min / \$max	Minimum / maximum value in the group.
\$first / \$last	First / last value in the group (use after \$sort).
\$push	Build an array of all values in the group.
\$addToSet	Build an array of unique values.
\$count	Count documents in the group (newer syntax).

26.4 \$project — Reshaping Documents

```
db.users.aggregate([
  { $project: {
    _id: 0,
    fullName: { $concat: ['$firstName', ' ', '$lastName'] },
    ageGroup: {
      $cond: {
        if: { $gte: ['$age', 18] },
        then: 'adult',
        else: 'minor'
      }
    },
    emailDomain: {
      $arrayElemAt: [{ $split: ['$email', '@'] }, 1]
    }
  }}
]);
```

26.5 \$unwind — Flatten Arrays

```
// Document: { _id, name, hobbies: ['chess', 'reading', 'hiking'] }
db.users.aggregate([
  { $unwind: '$hobbies' },
  { $group: {
    _id: '$hobbies',
    count: { $sum: 1 }
  }},
  { $sort: { count: -1 } }
]);

// Result: { _id: 'reading', count: 42 }, { _id: 'chess', count: 28 }, ...
```

26.6 \$lookup — Joining Collections

```
// Join orders with customers
db.orders.aggregate([
  { $lookup: {
    from: 'customers',           // collection to join
    localField: 'customerId',    // field in orders
    foreignField: '_id',        // field in customers
    as: 'customer'              // output array field
  }},
  { $unwind: '$customer' },      // flatten the joined array
  { $project: {
    total: 1,
    'customer.name': 1,
    'customer.email': 1
  }}
]);
```

NOTE: \$lookup is a left outer join. If no match is found, 'as' contains an empty array. Use \$unwind with preserveNullAndEmptyArrays: true to keep unmatched docs.

26.7 \$facet — Multi-Faceted Aggregation

```
// Get count, top products, and price stats in ONE query
db.products.aggregate([
  { $facet: {
    totalCount: [{ $count: 'count' }],
    topRated: [
      { $sort: { rating: -1 } },
      { $limit: 5 },
      { $project: { name: 1, rating: 1 } }
    ],
    priceStats: [
      { $group: {
        _id: null,
        avg: { $avg: '$price' },
        min: { $min: '$price' },
        max: { $max: '$price' }
      }}
    ]
  }}
]);
```

26.8 Performance Tips for Aggregation

- Place \$match as early as possible — it reduces the working set for later stages.
- \$match + \$sort early can use indexes; later stages cannot.
- Use \$project early to remove unneeded fields and reduce memory.
- \$lookup can be slow on large collections — index the foreignField.
- Pipeline stages have a 100MB memory limit; use { allowDiskUse: true } for large datasets.
- Use explain('executionStats') to see how the pipeline executes.

Chapter 27: Schema Design Patterns

MongoDB is schema-flexible, but good schema design is critical for performance and scalability. The key decision in document database design is: embed or reference?

27.1 Embedding vs Referencing

Aspect	Embed	Reference
Pattern	Nested documents/arrays	Store ObjectId, query separately
Read performance	Single query — very fast	Requires \$lookup or extra query
Update overhead	Update entire doc	Update only related doc
Data duplication	Possible	Avoided
Best for	Contains/owns relationships	Many-to-many, large sub-docs
Document size	Watch the 16MB limit	No size concerns

27.2 When to Embed

- The child entity has no independent meaning (e.g., address inside user).
- Data is always accessed together (one-to-few relationships).
- The contained data doesn't grow unboundedly.
- Strong containment / ownership (post → comments on a blog).
- You need atomic updates on the parent and child together.

27.3 When to Reference

- Many-to-many relationships (e.g., students ↔ courses).
- The referenced entity is large or frequently updated independently.
- The relationship is one-to-very-many (millions of items).
- The child entity is queried independently of the parent.
- Document would exceed the 16MB BSON size limit.

27.4 One-to-One

```
// EMBEDDED (preferred for one-to-one)
{
  _id: 1,
  name: 'Alice',
  address: {
    street: '123 Main St',
    city: 'Chennai',
    zip: '600001'
  }
}

// REFERENCED (when address is huge or shared)
// users: { _id: 1, name: 'Alice', addressId: 99 }
// addresses: { _id: 99, street: '...', city: '...' }
```

27.5 One-to-Many

```
// One-to-FEW: embed array
{
  _id: 1,
  name: 'Blog Post',
  comments: [
    { user: 'Alice', text: 'Great!', date: ISODate() },
    { user: 'Bob', text: 'Helpful', date: ISODate() }
  ]
}

// One-to-MANY: reference (parent stores child IDs)
// blogs: { _id: 1, title: '...', commentIds: [101, 102] }
// comments: { _id: 101, blogId: 1, text: '...' }

// One-to-SQUILLIONS: child stores parent reference
// devices: { _id: 1, name: 'sensor-a' }
// readings: { _id: 901, deviceId: 1, value: 23.4, ts: ISODate() }
```

27.6 Many-to-Many

```
// Use two-way reference arrays for many-to-many
// students: {
//   _id: 1, name: 'Alice', courseIds: [10, 11, 12]
// }
// courses: {
//   _id: 10, title: 'Algorithms', studentIds: [1, 2, 3]
// }

// Query students in a course:
db.students.find({ courseIds: 10 });

// Query courses for a student:
db.courses.find({ studentIds: 1 });
```


27.7 Common Design Patterns

Pattern	Use Case
Bucket	Group time-series data into buckets (e.g., 1 doc per hour with array of readings).
Computed	Pre-compute aggregations (e.g., totalLikes field instead of counting).
Subset	Store frequently accessed subset in main doc, rest in separate collection.
Extended Reference	Duplicate frequently-needed fields from referenced doc.
Outlier	Handle outliers (e.g., celebrity with millions of followers) specially.
Schema Versioning	Add 'schema_version' field to support gradual migration.
Tree (Parent Refs)	Each doc stores parentId for hierarchical data.
Tree (Materialized Path)	Store full path as string (e.g., '/root/a/b/c').

IMPORTANT: MongoDB favors denormalization. Unlike SQL, duplicating data for read performance is often the right choice — disk is cheap, joins are expensive.

Chapter 28: Mongoose ODM

Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js. It provides schema definition, validation, type casting, query building, middleware (hooks), and more — making it the most popular way to use MongoDB with Node.js applications.

28.1 Installation and Connection

```
// npm install mongoose

const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/myapp')
  .then(() => console.log('Connected to MongoDB'))
  .catch(err => console.error('Connection error:', err));

// Event listeners
mongoose.connection.on('error', err => console.error(err));
mongoose.connection.on('disconnected', () => console.log('Disconnected'));
```

28.2 Defining a Schema

```
const { Schema } = mongoose;

const userSchema = new Schema({
  name: { type: String, required: true, trim: true },
  email: {
    type: String, required: true, unique: true,
    lowercase: true,
    match: [/\.+@.+\.+/, 'Invalid email']
  },
  age: { type: Number, min: 0, max: 150 },
  role: { type: String, enum: ['user', 'admin'], default: 'user' },
  hobbies: [String],
  address: {
    street: String,
    city: String
  },
  createdAt: { type: Date, default: Date.now }
}, { timestamps: true }); // adds createdAt + updatedAt

const User = mongoose.model('User', userSchema);
```

28.3 Schema Types

Type	Description
String	UTF-8 text. Modifiers: trim, lowercase, uppercase, match.
Number	Numeric. Modifiers: min, max.
Date	Date object. Modifiers: min, max.
Boolean	true/false.
Buffer	Binary data.
ObjectId	Reference to another doc: Schema.Types.ObjectId.
Array	Array of any type: [String], [Number], [SubSchema].
Map	Key-value map with typed values.
Mixed	Anything goes: Schema.Types.Mixed.

28.4 CRUD with Mongoose

```
// CREATE
const user = new User({ name: 'Alice', email: 'a@b.com' });
await user.save();
// or
await User.create({ name: 'Bob', email: 'b@c.com' });

// READ
const all = await User.find();
const adults = await User.find({ age: { $gte: 18 } });
const one = await User.findOne({ email: 'a@b.com' });
const byId = await User.findById('507f1f77bcf86cd799439011');

// UPDATE
await User.updateOne({ _id: id }, { age: 30 });
const updated = await User.findByIdAndUpdate(id, { age: 31 }, { new: true });

// DELETE
await User.deleteOne({ _id: id });
await User.findByIdAndDelete(id);
```

28.5 Validation

```
// Built-in validators
const schema = new Schema({
  name: { type: String, required: [true, 'Name is required'] },
  age: { type: Number, min: 0, max: 120 },
  role: { type: String, enum: ['user', 'admin', 'guest'] }
});

// Custom validator
schema.path('email').validate(function(value) {
  return /\S+@\S+\.\S+/.test(value);
}, 'Invalid email format');

// Async validation
schema.path('username').validate(async function(v) {
  const count = await this.constructor.countDocuments({ username: v });
  return count === 0;
}, 'Username already taken');
```

28.6 Middleware (Hooks)

```
// Pre/post hooks run before/after operations

userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});

userSchema.post('save', function(doc, next) {
  console.log('Saved:', doc._id);
  next();
});

// Query middleware
userSchema.pre('find', function() {
  this.where({ deleted: { $ne: true } }); // soft-delete filter
});
```

28.7 Virtuals

```
// Virtuals are properties not stored in MongoDB
userSchema.virtual('fullName').get(function() {
  return `${this.firstName} ${this.lastName}`;
});

userSchema.virtual('fullName').set(function(v) {
  [this.firstName, this.lastName] = v.split(' ');
});

// Include virtuals in JSON output
userSchema.set('toJSON', { virtuals: true });
```

28.8 Populate (Reference Joining)

```
// Schema with ObjectId reference
const postSchema = new Schema({
  title: String,
  author: { type: Schema.Types.ObjectId, ref: 'User' },
  comments: [{ type: Schema.Types.ObjectId, ref: 'Comment' }]
});

// Populate replaces ObjectIds with the actual documents
const post = await Post.findById(id)
  .populate('author', 'name email')      // only name and email
  .populate({
    path: 'comments',
    populate: { path: 'author' }          // nested populate
  });

console.log(post.author.name); // not just an ObjectId now
```

28.9 Instance and Static Methods

```
// Instance method (on a document)
userSchema.methods.comparePassword = async function(pwd) {
  return bcrypt.compare(pwd, this.password);
};

// Static method (on the model)
userSchema.statics.findByEmail = function(email) {
  return this.findOne({ email });
};

// Query helper
userSchema.query.byRole = function(role) {
  return this.where({ role });
};

// Usage:
const u = await User.findByEmail('a@b.com');
await u.comparePassword('secret');
const admins = await User.find().byRole('admin');
```

TIP: Use `lean()` for read-heavy queries when you don't need Mongoose documents: `User.find().lean()` returns plain JS objects, ~10x faster and ~5x less memory.

Chapter 29: Transactions

MongoDB supports multi-document ACID transactions (since 4.0 on replica sets, 4.2 on sharded clusters). Transactions allow multiple operations across multiple documents and collections to succeed or fail as a single unit.

NOTE: Single-document operations in MongoDB are always atomic. You only need transactions when you must update multiple documents/collections atomically.

29.1 ACID in MongoDB

Property	Meaning
Atomicity	All operations in transaction succeed or all roll back.
Consistency	Database moves from one valid state to another.
Isolation	Concurrent transactions don't see each other's intermediate state.
Durability	Committed transactions persist even after crashes.

29.2 Native MongoDB Driver — Transaction Example

```
const session = client.startSession();

try {
  session.startTransaction();

  const accounts = client.db('bank').collection('accounts');
  await accounts.updateOne(
    { _id: 'alice' },
    { $inc: { balance: -100 } },
    { session }
  );
  await accounts.updateOne(
    { _id: 'bob' },
    { $inc: { balance: 100 } },
    { session }
  );

  await session.commitTransaction();
} catch (err) {
  await session.abortTransaction();
  throw err;
} finally {
  await session.endSession();
}
```

29.3 Mongoose Transactions

```
const session = await mongoose.startSession();
session.startTransaction();

try {
  await Account.updateOne(
    { _id: aliceId },
    { $inc: { balance: -100 } }
  ).session(session);

  await Account.updateOne(
    { _id: bobId },
    { $inc: { balance: 100 } }
  ).session(session);

  await session.commitTransaction();
} catch (e) {
  await session.abortTransaction();
  throw e;
} finally {
  session.endSession();
}

// Or use the helper:
await mongoose.connection.transaction(async (session) => {
  await Account.updateOne({ _id: a }, { $inc: { balance: -100 } }).session(session);
  await Account.updateOne({ _id: b }, { $inc: { balance: 100 } }).session(session);
});
```

29.4 Transaction Best Practices

- Keep transactions short — they hold locks and impede concurrency.
- Limit to fewer than 1000 documents modified per transaction.
- Default transaction timeout is 60 seconds.
- Transactions require a replica set or sharded cluster (not standalone).
- Don't use transactions if a single-document update can achieve the same result.
- Handle `TransientTransactionError` by retrying — MongoDB recommends retry loops.
- Avoid using transactions for analytics or batch processing — they're for OLTP.

Chapter 30: Replication and Sharding

MongoDB scales horizontally through two main mechanisms: replication (for high availability and read scaling) and sharding (for write scaling and storing data larger than a single server).

30.1 Replication — Replica Sets

A replica set is a group of MongoDB processes that maintain the same data. It provides redundancy, high availability, and disaster recovery. A typical replica set has at least 3 nodes: one PRIMARY and two SECONDARY nodes.

Role	Description
Primary	Receives all write operations. Only one primary at a time.
Secondary	Replicates the primary's oplog. Can serve reads.
Arbiter	Votes in elections but holds no data. Used to break ties.
Hidden	Replicates data but is invisible to applications.
Delayed	Replicates data with a delay — used for backups against operator error.

30.2 How Replication Works

1. All writes go to the PRIMARY, which records them in its oplog.
2. SECONDARIES asynchronously pull oplog entries and apply them.
3. If the primary becomes unavailable, an election picks a new primary.
4. An election requires a majority of voting members to be reachable.
5. Clients automatically reconnect to the new primary.

30.3 Write Concern


```
// Write concern specifies how many nodes must acknowledge writes

// w: 1 (default) - primary only
db.users.insertOne({ name: 'A' }, { writeConcern: { w: 1 } });

// w: 'majority' - majority of voting members (safest)
db.users.insertOne({ name: 'A' }, {
  writeConcern: { w: 'majority', j: true, wtimeout: 5000 }
});

// w: 0 - fire and forget (no ack, fastest but unsafe)
// j: true - require journal commit before ack
// wtimeout - max ms to wait for ack
```

30.4 Read Preference

Mode	Description
primary (default)	Read only from primary. Most consistent.
primaryPreferred	Primary if available, otherwise secondary.
secondary	Read only from secondaries.
secondaryPreferred	Secondary if available, otherwise primary.
nearest	Read from member with lowest network latency.

WARNING: Reading from secondaries can return STALE data because replication is asynchronous. Use only when slight staleness is acceptable.

30.5 Sharding

Sharding distributes data across multiple machines, supporting deployments with very large data sets and high write throughput. MongoDB shards data at the collection level using a shard key.

Component	Description
Shard	Each shard is itself a replica set, holding a subset of the data.
mongos	Query router; clients connect to mongos instead of individual shards.
Config Server	Replica set storing cluster metadata and shard configuration.
Shard Key	Field(s) used to distribute documents among shards.
Chunk	Contiguous range of shard key values; MongoDB balances chunks.

30.6 Choosing a Shard Key

- High cardinality — many possible values.
- Low frequency — no single value dominates.
- Non-monotonic — avoid keys like timestamps that always increase (creates hotspots).
- Match query patterns — ideally the shard key is in your common queries.
- Once chosen, the shard key cannot be changed easily — choose carefully.
- Hashed shard keys (MongoDB hashes the value) distribute writes evenly.

30.7 Sharding Strategies

Strategy	When to Use
Ranged	Range-based queries are common; data has natural order.
Hashed	Want even distribution; monotonic keys (timestamps, ObjectIds).
Zoned	Geographic or compliance requirements (data residency).

Chapter 31: Performance Optimization

MongoDB performance depends on schema design, indexes, queries, hardware, and deployment topology. This chapter summarizes the most effective tuning techniques.

31.1 Query Optimization

- Use `explain('executionStats')` to inspect query plans.
- Aim for `IXSCAN`, not `COLLSCAN`, on production queries.
- Use `projection` to return only needed fields — reduces network/memory.
- Use `limit()` and pagination — avoid loading entire collections.
- Avoid negation operators (`$ne`, `$nin`) where possible — they can't use indexes efficiently.
- Avoid regex queries that don't anchor to the start (`/^prefix/` uses indexes; `/substring/` does not).

31.2 Indexing Best Practices

```
// 1. ESR rule for compound indexes: Equality, Sort, Range
db.orders.createIndex({
  status: 1,          // Equality
  createdAt: -1,      // Sort
  total: 1            // Range
});

// 2. Covered queries (all fields in index)
db.users.createIndex({ email: 1, name: 1 });
db.users.find(
  { email: 'a@b.com' },
  { _id: 0, email: 1, name: 1 } // covered!
);

// 3. Drop unused indexes (they slow down writes)
db.collection.aggregate([ { $indexStats: {} } ]);
```

31.3 Schema Optimization

- Embed data accessed together; reference data accessed independently.
- Use short field names in very large collections — they reduce document size.
- Avoid documents with unbounded array growth.
- Use the Bucket pattern for time-series data.
- Pre-compute aggregations when the cost of computing on read is too high.

31.4 Hardware and Configuration

- Working set (frequently accessed data + indexes) should fit in RAM.
- Use SSDs over HDDs — random I/O dominates.
- Use the WiredTiger storage engine (default since 3.2).
- Tune WiredTiger cache: usually 50% of RAM minus 1GB.
- Use connection pooling on clients (Mongoose defaults to 5 — increase for high load).
- Use replica sets to distribute reads (with appropriate read preferences).

31.5 Monitoring Tools

Tool	Use
mongostat	Real-time database operations counters.
mongotop	Read/write activity per collection.
explain()	Query plan and execution stats.
currentOp()	View currently running operations.
MongoDB Atlas / Compass	GUI monitoring, performance advisor.
Database Profiler	Logs slow queries: <code>db.setProfilingLevel(1, 100)</code> .

TIP: The MongoDB Atlas Performance Advisor auto-suggests indexes based on actual workload. For self-hosted deployments, look for COLLSCAN entries in slow logs.

31.6 Common Performance Anti-Patterns

Anti-Pattern	Fix
Massive arrays in documents	Use the Bucket or reference pattern.
Bloated documents (16MB limit)	Move infrequently-accessed fields to a separate collection.
Unbounded \$lookup	Add indexes on foreignField; consider denormalization.
Many small queries (N+1)	Use \$lookup or batch fetching.
Indexing every field	Each index slows writes; index only what queries need.
Using ObjectId.timestamp for time queries	Use an explicit indexed date field.

PART IV

Interview Questions & Answers

Interview Preparation Guide

This section consolidates the most frequently asked Node.js, Express.js, and MongoDB interview questions for Associate Software Engineer positions (L1 and L2 levels). Questions are organized by technology and difficulty.

NOTE: L1 (Junior Associate) — fundamentals, definitions, basic usage. Expect ~15–20 questions in a typical interview.

TIP: L2 (Associate / Mid-level) — internals, trade-offs, scaling, debugging. Expect scenario-based questions and live coding/debugging.

IMPORTANT: Always structure your answer: **Definition** → **How it works** → **Example** → **Trade-offs**. Interviewers value clarity and depth over memorized lines.

Node.js — L1 (Fundamental Questions)

Q1. What is Node.js? How is it different from JavaScript in the browser?

Node.js is a JavaScript runtime built on Chrome's V8 engine that runs JavaScript outside the browser. Unlike browser JS (which has DOM, window, fetch), Node.js provides server-side APIs: file system (fs), networking (net, http), child processes, streams, buffers, and OS access. It uses an event-driven, non-blocking I/O model that makes it efficient for I/O-heavy applications like APIs and real-time servers.

Q2. What is the V8 engine?

V8 is Google's open-source JavaScript engine written in C++. It compiles JavaScript directly to machine code (just-in-time compilation) instead of interpreting it, which is why JS in Node.js / Chrome runs so fast. V8 also handles memory management with garbage collection.

Q3. Is Node.js single-threaded or multi-threaded?

Node.js is **single-threaded for JavaScript execution** — your code runs on one main thread (the event loop). However, libuv (Node's underlying library) maintains a thread pool (default 4 threads) for I/O operations like file system access, DNS lookups, and crypto functions. So Node is technically multi-threaded under the hood, but the developer-facing model is single-threaded.

Q4. What is the event loop?

The event loop is Node's core mechanism for handling asynchronous operations on a single thread. It continuously checks the call stack and various queues, processing callbacks when the stack is empty. The loop runs in phases: Timers → Pending Callbacks → Idle/Prepare → Poll → Check → Close Callbacks. Microtasks (Promises, process.nextTick) run between each phase.

Q5. What is the difference between process.nextTick() and setImmediate()??

process.nextTick() callbacks run immediately after the current operation, before the event loop continues — they have the highest priority and can starve I/O if overused. **setImmediate()** callbacks run in the Check phase, after I/O polling. process.nextTick fires before setImmediate.

Q6. What is the difference between setTimeout(fn, 0) and setImmediate(fn)?

Within an I/O cycle, setImmediate always runs first. Outside an I/O cycle (e.g., in the main module), their order is non-deterministic because setTimeout(0) is actually treated as setTimeout(1) by Node, and timing varies.

Q7. What are modules in Node.js? Explain CommonJS vs ES Modules.

Modules are reusable units of code. **CommonJS** (require/module.exports) is Node's original synchronous module system. **ES Modules** (import/export) is the standardized JavaScript module system supported in Node 14+. CommonJS loads synchronously and dynamically; ES Modules load asynchronously and statically. To use ES Modules, set "type": "module" in package.json or use .mjs extension.

Q8. What is package.json? What is package-lock.json?

package.json defines project metadata: name, version, dependencies, scripts, main file, and configuration. **package-lock.json** locks the exact resolved versions of every dependency (and their dependencies) to ensure reproducible installs across machines. Always commit package-lock.json.

Q9. What is npm? What are some common npm commands?

npm (Node Package Manager) is the default package manager for Node.js. Common commands: *npm init* (create package.json), *npm install [pkg]* (install), *npm install --save-dev [pkg]* (dev dependency), *npm uninstall [pkg]*, *npm update*, *npm run [script]*, *npm outdated*, *npm audit*.

Q10. What is the difference between dependencies and devDependencies?

dependencies are required for the application to run in production (express, mongoose, etc.). **devDependencies** are only used during development (jest, eslint, nodemon, typescript). When you run *npm install --production* or set `NODE_ENV=production`, devDependencies are skipped.

Q11. What is a callback function? What is callback hell?

A callback is a function passed as an argument to another function, invoked when an async operation completes. **Callback hell** is deeply nested callbacks ("pyramid of doom") making code hard to read and maintain. Solutions: Promises, async/await, named functions, or modularization.

Q12. What is a Promise? What are its states?

A Promise is an object representing the eventual completion (or failure) of an async operation. Three states: **pending** (initial), **fulfilled** (resolved with value), **rejected** (failed with reason). Once settled (fulfilled/rejected), state cannot change.

Q13. What is async/await?

async/await is syntactic sugar over Promises (ES2017). An *async* function always returns a Promise. *await* pauses execution until a Promise settles, making async code read like sync code. Errors are caught with try/catch.

Q14. What is the difference between Promise.all and Promise.allSettled?

Promise.all resolves when ALL promises resolve, or rejects as soon as ANY rejects. **Promise.allSettled** waits for ALL promises to settle (fulfilled or rejected) and returns an array of result objects with status. Use allSettled when you need every result regardless of individual failures.

Q15. What is an EventEmitter?

EventEmitter is a core Node.js class that implements the publisher-subscriber pattern. Objects emit named events that call listener functions. Methods: *on()*, *emit()*, *once()*, *removeListener()*. Many Node objects (streams, HTTP servers, processes) inherit from EventEmitter.

Q16. What are streams in Node.js?

Streams are objects that read or write data in chunks, allowing efficient handling of large data without loading it entirely into memory. Four types: **Readable** (fs.createReadStream), **Writable** (fs.createWriteStream), **Duplex** (both, like TCP sockets), **Transform** (modifies data, like zlib).

Q17. What is a Buffer?

Buffer is Node's class for handling raw binary data. It's a fixed-size chunk of memory allocated outside V8's heap. Used for file I/O, network protocols, and binary data manipulation. Created with *Buffer.from()*, *Buffer.alloc()*, *Buffer.allocUnsafe()*.

Q18. What is `__dirname` and `__filename`?

`__dirname` is the absolute path of the directory containing the current module file. `__filename` is the absolute path of the current module file itself. Available in CommonJS; in ES Modules, use *import.meta.url* and convert via *fileURLToPath()*.

Q19. What is the difference between `exports` and `module.exports`?

`module.exports` is what's actually returned when *require()* is called. `exports` is a reference to `module.exports`. You can add properties to *exports.foo = ...*, but reassigning *exports = ...* breaks the reference. Use *module.exports = ...* to export a single value (class, function, etc.).

Q20. What is the global object in Node.js?

`global` is the global namespace object in Node.js (equivalent to *window* in browsers). Variables declared with *var* at the top level of a module are NOT added to global (modules have their own scope), unlike browser scripts.

Q21. How do you handle errors in Node.js?

Multiple strategies: (1) **try/catch** for sync code and *async/await*. (2) **Error-first callbacks** — *callback(err, result)*. (3) **Promise .catch()**. (4) **EventEmitter 'error' events** — always handle them. (5) Global handlers: *process.on('uncaughtException')* and *process.on('unhandledRejection')*.

Q22. What is `process.env`? How do you use environment variables?

`process.env` is an object containing all environment variables. Used for configuration that varies between environments (dev, staging, prod): *process.env.DATABASE_URL*, *process.env.PORT*. Use the *dotenv* package to load variables from a *.env* file. Never commit *.env* to version control.

Q23. What is REPL in Node.js?

REPL (Read-Eval-Print-Loop) is the interactive shell you get by running *node* in the terminal. Useful for quick experiments, debugging snippets, and exploring APIs. Supports auto-completion (Tab) and special commands like *.help*, *.save*, *.load*.

Q24. What are the differences between `fs.readFile` and `fs.readFileSync`?

`fs.readFile` is asynchronous and non-blocking; it uses a callback (or returns a Promise with *fs.promises*). `fs.readFileSync` blocks the event loop until the file is read. Sync versions are fine for startup-only code (loading config files) but should NEVER be used in request handlers.

Q25. What is libuv?

libuv is a C library that provides Node.js's asynchronous I/O capabilities. It implements the event loop, thread pool, and abstractions over OS-specific I/O (epoll on Linux, kqueue on macOS, IOCP on Windows). Anything in Node that's async uses libuv under the hood.

Node.js — L2 (Advanced Questions)

Q26. Explain the phases of the event loop in detail.

Six phases: (1) **Timers** — executes `setTimeout/setInterval` callbacks whose threshold has elapsed. (2) **Pending Callbacks** — executes deferred I/O callbacks (e.g., TCP errors). (3) **Idle/Prepare** — internal only. (4) **Poll** — retrieves new I/O events, executes I/O callbacks. Blocks if no immediate work. (5) **Check** — executes `setImmediate` callbacks. (6) **Close Callbacks** — like `socket.on('close')`. Between every phase, the microtask queues (`process.nextTick` → `Promise.then`) are drained.

Q27. How does the thread pool in Node.js work?

libuv maintains a thread pool (default 4, max 1024) used for operations that can't be done asynchronously at the OS level: file system operations on most platforms, DNS lookups (`dns.lookup`), some crypto operations (`pbkdf2`, `randomBytes`), and `zlib`. Increase via `UV_THREADPOOL_SIZE` environment variable. CPU-bound work in these operations can starve other I/O.

Q28. How would you handle CPU-intensive tasks in Node.js?

Options: (1) **worker_threads** module — true parallel JS execution in separate threads (best for CPU-bound JS work). (2) **child_process** (`fork`) — spawn a separate Node process (more memory but isolated). (3) **cluster** module — fork multiple Node processes for I/O scaling. (4) Offload to external service (queue + worker). (5) Use native addons (C++) via N-API.

Q29. What is the difference between cluster and worker_threads?

cluster forks multiple Node PROCESSES (separate memory). Best for scaling I/O-bound apps across CPU cores; processes communicate via IPC. **worker_threads** creates multiple THREADS in the same process, sharing memory via `SharedArrayBuffer`. Best for CPU-bound JS computation. Cluster has higher memory overhead but better isolation; `worker_threads` has lower overhead but requires careful concurrency handling.

Q30. How do you detect and fix memory leaks in Node.js?

Detection: (1) Monitor `process.memoryUsage()` over time — growing `heapUsed` indicates a leak. (2) Use `--inspect` with Chrome DevTools to take heap snapshots and compare. (3) Tools: `clinic.js`, `node-memwatch`, `heapdump`. Common causes: closures holding references, forgotten event listeners, global variables, infinite caches, timers not cleared. Fix by removing unintended references.

Q31. What is backpressure in streams? How do you handle it?

Backpressure occurs when the writable side of a stream can't consume data as fast as the readable side produces it, causing the internal buffer to grow. Without handling, you can exhaust memory. Solutions: (1) Use `.pipe()` or `stream.pipeline()` — they handle backpressure automatically. (2) Check the return value of `write()` — if false, wait for 'drain' event before writing more.

Q32. What is the use of `stream.pipeline()` over `.pipe()`?

`stream.pipeline()` chains streams and automatically forwards errors and cleans up resources when any stream fails or completes. **`.pipe()`** does NOT forward errors — you must handle 'error' on each stream manually, otherwise a failure can leak file descriptors. `pipeline()` is the modern, safer choice.

Q33. How does `require()` work internally?

When `require(id)` is called: (1) Resolve the id to a file path (Node's module resolution algorithm — core, `node_modules`, etc.). (2) Check the cache — if loaded, return cached exports. (3) Create a new Module object. (4) Read the file. (5) Wrap code in a function: `(exports, require, module, __filename, __dirname) => {...}`. (6) Execute the wrapper. (7) Cache and return `module.exports`.

Q34. What is the module wrapper function?

Before execution, Node wraps every module's code inside a function: `(function(exports, require, module, __filename, __dirname) { /* code */ });`. This gives each module its own scope (avoiding polluting global), provides the module-local variables, and lets Node hook into module loading. It's why `this` at the top level of a module equals `module.exports`, not global.

Q35. What are circular dependencies? How does Node handle them?

Circular dependency: module A requires B, and B requires A. Node handles it by returning a partial copy of `module.exports` if a module is required while still being loaded. This can lead to bugs where one module sees an empty object. Solution: restructure code to break the cycle, use lazy `require` inside functions, or extract shared logic to a third module.

Q36. How would you scale a Node.js application?

Multiple dimensions: (1) **Vertically** — bigger machine. (2) **Horizontally** — multiple instances behind a load balancer. (3) **Cluster module / PM2** — use all CPU cores on one machine. (4) **Microservices** — split into smaller services. (5) **Caching** — Redis for hot data. (6) **Database scaling** — read replicas, sharding. (7) **Queue heavy work** — RabbitMQ/SQS with workers. (8) **CDN** for static assets.

Q37. What is the difference between `exec`, `spawn`, and `fork`?

All from `child_process`: **`exec`** — runs a command in a shell, buffers entire output (use for short commands). **`spawn`** — runs a command without a shell, streams output (use for long-running processes or large output). **`fork`** — special case of `spawn` for Node scripts; sets up IPC channel for communication.

Q38. What is `process.nextTick` starvation?

Because `process.nextTick` callbacks execute before the event loop continues, recursively calling `nextTick` can prevent the loop from ever reaching I/O or timers — "starving" them. Symptoms: HTTP server stops responding while CPU is busy. Solution: use `setImmediate` for recursive deferral instead.

Q39. What is graceful shutdown?

Graceful shutdown handles SIGTERM/SIGINT signals: (1) Stop accepting new connections. (2) Wait for in-flight requests to finish. (3) Close database connections. (4) Clean up resources. (5) Exit. Important for zero-downtime deployments and avoiding data corruption. Implement via `process.on('SIGTERM', ...)` and `server.close(callback)`.

Q40. How does `require.cache` work and when would you clear it?

`require.cache` is an object mapping resolved module paths to their cached Module instances. Subsequent `require()` calls return cached exports. You might clear it for hot-reloading (delete `require.cache[path]`) but this is risky in production — references to old exports still exist, causing inconsistencies. Better to use proper dev tools (nodemon) or restart the process.

Q41. What's the difference between `Buffer.alloc()` and `Buffer.allocUnsafe()`?

`Buffer.alloc(size)` creates a buffer of size bytes filled with zeros — safe but slower. **`Buffer.allocUnsafe(size)`** creates an uninitialized buffer — faster but may contain old memory contents (potential data leak). Use `allocUnsafe` only when you'll immediately overwrite the entire buffer.

Q42. Explain microtasks vs macrotasks.

Macrotasks are processed in the event loop's phases: `setTimeout`, `setImmediate`, I/O callbacks, etc. **Microtasks** are processed between macrotasks: `process.nextTick` (highest priority) and Promise callbacks (`.then`, `.catch`, `.finally`). After each macrotask, the microtask queue is fully drained before the next macrotask runs.

Express.js — L1 (Fundamental Questions)

Q43. What is Express.js?

Express is a minimal, unopinionated web framework for Node.js. It provides routing, middleware support, request/response handling, and templating — making it easy to build APIs and web servers. It's the de-facto standard Node web framework and the foundation for many others (NestJS, Sails, etc.).

Q44. What is middleware in Express?

Middleware is a function with signature *(req, res, next)* that has access to the request and response objects and the next middleware in the cycle. It can execute code, modify req/res, end the request, or call next() to pass control. Used for logging, authentication, parsing, error handling, etc.

Q45. What are the types of middleware in Express?

Five types: (1) **Application-level** — `app.use()`, `app.METHOD()`. (2) **Router-level** — same as app-level but on Router instance. (3) **Built-in** — `express.json()`, `express.urlencoded()`, `express.static()`. (4) **Third-party** — morgan, helmet, cors. (5) **Error-handling** — four arguments (err, req, res, next).

Q46. What is routing in Express?

Routing determines how an application responds to client requests based on URL and HTTP method. Defined via `app.METHOD(path, handler)` — `app.get()`, `app.post()`, `app.put()`, `app.delete()`, etc. Supports route parameters (`:id`), query strings, regex paths, and modular routing via `express.Router()`.

Q47. What's the difference between `app.use()` and `app.get()`?

`app.use([path], middleware)` mounts middleware for ALL HTTP methods at the given path (defaults to '/'). It's used for middleware functions and matches any path that STARTS with the given prefix. **`app.get(path, handler)`** only matches GET requests and requires an EXACT path match (including the params).

Q48. What is `req.params`, `req.query`, and `req.body`?

`req.params` — route parameters from the URL path (e.g., `/users/:id` → `req.params.id`). **`req.query`** — parsed query string (e.g., `/users?role=admin` → `req.query.role`). **`req.body`** — parsed request body (requires body-parsing middleware like `express.json()` for JSON or `express.urlencoded()` for forms).

Q49. How do you send a JSON response in Express?

Use `res.json(object)`. It automatically sets Content-Type to `application/json` and stringifies the object. Optionally chain status: `res.status(201).json({...})`. `res.send()` can also send JSON but `res.json` is explicit and handles edge cases like null/undefined properly.

Q50. What is `express.Router`?

`express.Router()` creates a modular, mountable route handler. It's like a mini Express app — supports its own middleware and routes. Used to split routes into separate files: define routes in `userRoutes.js`, then `app.use('/users', userRouter)` in the main file. Improves organization in larger apps.

Q51. How do you handle errors in Express?

(1) **Synchronous errors** — Express catches them automatically and passes to error handler. (2) **Async errors** — call `next(err)` or use async wrappers. (3) Define an error-handling middleware with 4 arguments: `(err, req, res, next)`, placed AFTER all other middleware. (4) Use `express-async-handler` or wrap async handlers to forward rejections.

Q52. What is CORS? How do you enable it in Express?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that restricts cross-origin HTTP requests. Without proper headers, browsers block frontend code from another origin from calling your API. Enable using the `cors` npm package: `app.use(cors({ origin: 'https://example.com' })))`.

Q53. How do you serve static files in Express?

Use the built-in `express.static` middleware: `app.use(express.static('public'))`. Files in the public directory become accessible at the root URL (`public/style.css` → `/style.css`). You can mount under a path: `app.use('/static', express.static('public'))`.

Q54. What is the order of middleware important?

Yes — critical. Middleware executes in the order it's registered. Body parsers must come before routes that read `req.body`. Authentication middleware must come before protected routes. Error handlers must come LAST. `app.use()` calls registered after a route never run for that route.

Q55. What HTTP status codes do you commonly use?

200 OK, **201** Created, **204** No Content, **301** Moved Permanently, **302** Found, **304** Not Modified, **400** Bad Request, **401** Unauthorized, **403** Forbidden, **404** Not Found, **409** Conflict, **422** Unprocessable Entity, **429** Too Many Requests, **500** Internal Server Error, **503** Service Unavailable.

Q56. What's the difference between PUT and PATCH?

PUT replaces the entire resource — the request body should contain the complete updated resource. **PATCH** applies a partial update — only the fields included in the body are modified. PUT is idempotent and replaces; PATCH is partial.

Q57. How do you read environment variables in Express?

Use `process.env.VARIABLE_NAME`. To load from a `.env` file in development, install the `dotenv` package and call `require('dotenv').config()` at the top of your entry file. In production, set env vars via the deployment platform (Docker, Heroku, AWS, etc.).

Q58. What is body-parser? Is it still needed?

body-parser parses request bodies (JSON, URL-encoded). Since Express 4.16+, it's built into Express as `express.json()` and `express.urlencoded()`. You no longer need to install body-parser separately unless you need its advanced options or are using an older Express version.

Q59. How do you implement authentication in Express?

Common approaches: (1) **Sessions** — use express-session with a store like Redis or MongoDB. Cookie holds session ID. (2) **JWT (JSON Web Tokens)** — stateless tokens signed with a secret; sent in Authorization header. (3) **OAuth** — delegate to providers (Google, GitHub) via passport.js. (4) **API keys** for service-to-service auth. Always hash passwords with bcrypt.

Q60. What is the difference between res.send() and res.end()?

res.send(body) sends a response with optional body — handles strings, Buffers, objects (auto-JSON), and sets Content-Type/Content-Length headers. **res.end()** ends the response without data — used to terminate without a body (e.g., after 204 No Content). `res.send()` calls `res.end()` internally.

Express.js — L2 (Advanced Questions)

Q61. How would you structure a large Express application?

Layered architecture: (1) **routes/** — define endpoints, use Router. (2) **controllers/** — handle request/response logic. (3) **services/** — business logic. (4) **models/** — data access (Mongoose schemas). (5) **middleware/** — reusable middleware. (6) **utils/** — helpers. (7) **config/** — environment configuration. (8) **tests/**. Keep concerns separated; controllers thin, services rich.

Q62. How do you handle async errors in Express?

Express does NOT automatically catch errors thrown in async handlers (prior to Express 5). Options: (1) Wrap with `asyncHandler` utility: `fn => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next)`. (2) Use the `express-async-errors` package. (3) Use try/catch in every handler and call `next(err)`. (4) Upgrade to Express 5 which handles this natively.

Q63. JWT vs Sessions — which to use and when?

Sessions: server stores state; scales requires shared session store; easy to revoke; good for traditional web apps. **JWT:** stateless; scales horizontally without shared state; harder to revoke (needs blacklist or short expiry + refresh tokens); good for APIs/microservices/mobile. Use JWT for cross-domain or mobile, sessions for monolithic web apps with server-side rendering.

Q64. What are common Express security best practices?

(1) Use **helmet** for secure HTTP headers. (2) Enable **HTTPS** in production. (3) Rate limit endpoints (**express-rate-limit**). (4) Sanitize user input — **express-validator**, **express-mongo-sanitize**. (5) Use **cors** properly — don't use `origin: '*'` for authenticated APIs. (6) Hash passwords with **bcrypt**. (7) Set secure HTTP-only cookies. (8) Keep dependencies updated (npm audit). (9) Validate request schema. (10) Hide stack traces in production.

Q65. How does Express's middleware execution work internally?

Express maintains an array of middleware (the "stack") per Router. When a request arrives, Express iterates the stack, invoking each middleware whose path matches. Each receives `next` — calling `next()` moves to the next matching middleware, `next(err)` jumps to the next error-handling middleware, and calling neither `next()` nor responding hangs the request.

Q66. How do you implement role-based authorization (RBAC)?

Create a middleware factory: `requireRole(role) => (req, res, next) => { ... }`. After authentication populates `req.user`, the middleware checks user role against required role(s). Example: `router.delete('/users/:id', authenticate, requireRole('admin'), deleteUser)`. For complex authorization, use a library like Casbin or define permissions per resource.

Q67. How would you implement rate limiting?

Use **express-rate-limit** for simple in-memory limiting: `rateLimit({ windowMs: 15*60*1000, max: 100 })`. For distributed deployments, use a Redis-backed store (**rate-limit-redis**) so limits are shared across instances. Different routes can have different limits — stricter for login, looser for read-only. Consider sliding-window vs fixed-window algorithms.

Q68. How do you test Express applications?

Unit tests with Jest/Mocha for services and utilities. **Integration tests** with **supertest** for HTTP endpoints — `request(app).get('/api/users').expect(200)`. Use an in-memory database (mongodb-memory-server) or test database. Mock external services. Aim for tests that don't require the app to actually listen on a port (supertest can take the app object directly).

Q69. How do you implement file uploads?

Use **multer** middleware. Configure storage (disk or memory), file filters, and size limits. Single file: `upload.single('field')`; multiple files: `upload.array('files', maxCount)` or `upload.fields([...])`. For production, upload directly to S3 / cloud storage using multer-s3, or generate presigned URLs so files don't pass through your server.

Q70. What is the difference between app.locals and res.locals?

app.locals — properties available throughout the application's lifetime; ideal for app-wide config (site title, helpers). **res.locals** — properties scoped to a single request/response cycle; useful for passing data to views or downstream middleware (e.g., the authenticated user).

Q71. How would you implement API versioning?

Common strategies: (1) **URI versioning** — `/api/v1/users` (simple, explicit). (2) **Header versioning** — `Accept: application/vnd.app.v2+json` (cleaner URLs). (3) **Query parameter** — `/api/users?version=1` (least preferred). URI versioning is most common. Maintain backward compatibility, deprecate gracefully, and document the deprecation schedule.

Q72. How do you debug Express applications?

(1) Use `DEBUG=express:* node app.js` for verbose internal logs. (2) Use **node --inspect** with Chrome DevTools for breakpoints, heap snapshots, and profiling. (3) Use a logger (winston, pino) with appropriate levels. (4) Use morgan for HTTP request logs. (5) Add an error-handling middleware that logs full stack traces in dev. (6) Use APM tools (New Relic, Datadog) in production.

Q73. How do you scale an Express app across multiple cores?

Use the **cluster** module to fork one worker per CPU core, sharing the same port via the OS. Or use **PM2** in cluster mode: `pm2 start app.js -i max` — PM2 handles forking, monitoring, restarts, and zero-downtime reloads. For container deployments, run multiple container replicas behind a load balancer instead of cluster.

MongoDB — L1 (Fundamental Questions)

Q74. What is MongoDB?

MongoDB is a NoSQL, document-oriented database that stores data as flexible JSON-like documents (BSON format). It's schema-less, horizontally scalable, and designed for high availability and large unstructured datasets. Used widely for modern web applications, content management, real-time analytics, and IoT.

Q75. SQL vs NoSQL — what are the differences?

SQL: tabular schema, rows/columns, structured data, vertical scaling, ACID transactions, JOIN-based relationships, fixed schema (migrations). **NoSQL (MongoDB):** flexible documents, horizontal scaling, embedded data, BASE consistency by default (with ACID per document or via transactions), no fixed schema. SQL is best for relational/transactional data; NoSQL for unstructured/evolving/scale-out data.

Q76. What is a document in MongoDB?

A document is a BSON object — analogous to a row in SQL, but with nested structures. Each document has a unique `_id` field, can contain fields of any data type (string, number, array, embedded document, ObjectId, date, binary), and has a maximum size of 16MB.

Q77. What is a collection?

A collection is a group of documents — analogous to a table in SQL. Unlike SQL tables, collections don't enforce a schema by default, so documents can have different fields. Collections live inside a database; one MongoDB instance can host multiple databases.

Q78. What is BSON?

BSON (Binary JSON) is the binary-encoded serialization format MongoDB uses to store documents and exchange them over the wire. It extends JSON with additional types (ObjectId, Date, Binary, Decimal128, etc.) and is designed for efficient encoding/decoding and traversal.

Q79. What is the `_id` field?

Every MongoDB document has a unique `_id` field, which acts as the primary key. By default it's a 12-byte **ObjectId**: 4 bytes timestamp + 5 bytes random + 3 bytes counter. You can supply your own `_id` (any non-array value) when inserting, as long as it's unique within the collection.

Q80. What is an ObjectId?

ObjectId is MongoDB's default unique identifier — a 12-byte BSON type. It's designed to be globally unique without coordination, contains an embedded creation timestamp (extractable via `objectId.getTimestamp()`), and is naturally sortable by creation time. Always use ObjectId for relationships between documents.

Q81. What are the basic CRUD operations in MongoDB?

Create: insertOne, insertMany. **Read:** find, findOne, countDocuments. **Update:** updateOne, updateMany, replaceOne, findOneAndUpdate. **Delete:** deleteOne, deleteMany, findOneAndDelete. All return Promises in modern drivers and Mongoose.

Q82. What is the difference between find() and findOne()?

find() returns a **cursor** over all matching documents (must be iterated or converted to array via toArray()). **findOne()** returns a single document (the first match) directly, or null if not found. Use findOne when you only need one document — it's clearer and slightly more efficient.

Q83. What is an index? Why is it important?

An index is a data structure (B-tree) that improves query performance by allowing MongoDB to find documents without scanning the entire collection. Without an index, a query performs a COLLSCAN (full collection scan) — $O(n)$. With an index, it performs an IXSCAN — $O(\log n)$. Trade-off: indexes consume memory/disk and slow down writes.

Q84. What types of indexes does MongoDB support?

Single field, Compound (multi-field), Multikey (on array fields), Text (full-text search), 2dsphere (geospatial), Hashed (used for sharding), Wildcard (dynamic fields), TTL (time-to-live — auto-delete documents), Partial (only documents matching a filter), Unique (enforces uniqueness), Sparse (only documents that have the field).

Q85. What is the aggregation framework?

MongoDB's aggregation framework processes data through a pipeline of stages (\$match, \$group, \$project, \$sort, \$lookup, etc.) that transform documents and produce computed results. It's MongoDB's equivalent to SQL GROUP BY, JOIN, and window functions — but much more flexible.

Q86. What is the difference between embedded and referenced documents?

Embedded: nested document inside the parent (e.g., address inside user) — fast reads via single query, atomic updates of parent and child together. **Referenced:** child stored separately with an ObjectId pointing to it — requires \$lookup or extra query, but avoids duplication and bypasses the 16MB document size limit.

Q87. What does upsert mean?

Upsert = UPDATE + INSERT. With { upsert: true }, MongoDB updates the matching document if it exists, or inserts a new document with the filter criteria + update fields if no match exists. Useful for idempotent operations like "create or update".

Q88. What is the difference between \$set and \$push?

\$set: sets the value of a field (replaces existing). **\$push:** appends a value to an array field (creates the array if absent). Other array operators: \$pull (remove matching elements), \$addToSet (push only if not present), \$pop (remove first/last), \$each (push multiple).

Q89. What is Mongoose?

Mongoose is an ODM (Object Data Modeling) library for MongoDB and Node.js. It provides schema definition, validation, type casting, query building, middleware (hooks), virtuals, and populate (for joining references). It adds structure on top of MongoDB's flexibility.

Q90. What is `populate()` in Mongoose?

populate() replaces ObjectId references with the actual referenced documents. Define refs in your schema: `{ author: { type: ObjectId, ref: 'User' } }`, then query: `Post.find().populate('author')`. Equivalent to a SQL LEFT JOIN but performed as a separate query (or \$lookup under the hood).

Q91. How do you ensure uniqueness in MongoDB?

Create a unique index: `db.users.createIndex({ email: 1 }, { unique: true })`. Inserting a duplicate throws a duplicate key error (E11000). In Mongoose: `email: { type: String, unique: true }` creates the index automatically. Combine with `sparse: true` to allow null/missing values.

Q92. What is a TTL index?

Time-To-Live index — automatically deletes documents after a specified expiration. Create with `createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 })`. Useful for session data, cache entries, logs. MongoDB runs a background task every ~60 seconds to remove expired documents.

Q93. How do you paginate results in MongoDB?

Skip + limit: `find().skip((page-1) * 10).limit(10)` — simple but slow on large offsets because skip still scans. **Range-based / cursor pagination:** `find({ _id: { $gt: lastId } }).limit(10)` — much faster, uses indexes efficiently. Use cursor pagination for high-volume APIs.

Q94. What is the projection in a query?

Projection specifies which fields to return: `find({}, { name: 1, email: 1 })` returns only name and email (plus `_id`). Use `{ _id: 0 }` to exclude `_id`. Either include OR exclude — can't mix (except for `_id`). Projection reduces network and memory usage.

MongoDB — L2 (Advanced Questions)

Q95. How does indexing work internally in MongoDB?

MongoDB uses **B-tree indexes** stored separately from the data. Each index entry contains the indexed field's value and a pointer to the document. On query, MongoDB traverses the B-tree ($O(\log n)$) to find matching pointers, then fetches the actual documents. For covered queries (all required fields in the index), the document fetch is skipped entirely.

Q96. Explain the ESR rule for compound indexes.

ESR = Equality, Sort, Range: when designing compound indexes, put **Equality** fields first (those matched with exact values), then **Sort** fields, then **Range** fields. This minimizes documents examined and avoids in-memory sorting. Example: query filters by status (equality), sorts by createdAt, ranges on price → `{ status: 1, createdAt: -1, price: 1 }`.

Q97. What is a covered query?

A query where ALL fields needed (in filter AND projection) are present in a single index. MongoDB serves the result directly from the index without fetching the actual documents — much faster. Example: index on `{ email: 1, name: 1 }`, query `find({ email: x }, { _id: 0, email: 1, name: 1 })` is covered. Verify with `explain()` — `totalDocsExamined` should be 0.

Q98. How do you optimize a slow MongoDB query?

(1) Run `explain('executionStats')` — check stage (IXSCAN vs COLLSCAN), `totalDocsExamined`, `executionTimeMillis`. (2) Create an index that supports the query (apply the ESR rule). (3) Use projection to return only needed fields. (4) Use `limit()` to cap result size. (5) Avoid leading wildcards in regex. (6) Consider denormalization or pre-computed fields. (7) Re-evaluate the schema if many queries are slow.

Q99. Explain replication in MongoDB.

Replication uses **replica sets** — a group of MongoDB instances maintaining the same data. One PRIMARY accepts writes, SECONDARIES replicate the primary's **oplog** asynchronously. If the primary fails, an automatic election picks a new primary based on priority and data freshness. Provides high availability, fault tolerance, and read scaling.

Q100. What is the oplog?

The **oplog (operations log)** is a capped collection in the local database of every replica set member. It records every write operation. Secondaries pull and apply oplog entries to stay in sync. The oplog size determines how long a secondary can be offline before falling out of sync (default 5% of free disk).

Q101. What is sharding? When should you use it?

Sharding distributes a collection's data across multiple machines (shards), scaling writes and storage horizontally. Use it when: data exceeds single-server capacity, write throughput exceeds single-server limits, or working set exceeds available RAM. Not the first scaling step — try indexing, replication, and vertical scaling first.

Q102. How do you choose a shard key?

A good shard key has: (1) **High cardinality** — many distinct values. (2) **Low frequency** — no value dominates. (3) **Non-monotonic** — avoids hotspots (timestamps create them). (4) **Matches query patterns** — queries that include the shard key target a single shard. Once chosen, changing is difficult — design carefully.

Q103. What is write concern? Read concern?

Write concern specifies how many replica set members must acknowledge a write before it's confirmed (w: 1, w: 'majority', w: 0). Higher concerns = more durability, more latency. **Read concern** specifies the consistency level of read data ('local', 'available', 'majority', 'linearizable', 'snapshot'). 'majority' guarantees reading only committed data.

Q104. How do MongoDB transactions work?

Multi-document transactions (since v4.0 on replica sets) provide ACID guarantees across multiple operations, collections, and databases. Start a session, startTransaction(), perform operations passing the session, then commitTransaction() or abortTransaction(). Keep transactions short — default 60s timeout. Limit to <1000 modifications. Retry on TransientTransactionError.

Q105. What is the difference between \$lookup and populate?

\$lookup is a MongoDB aggregation stage that performs a left outer join with another collection — executed server-side in a single query. **populate** is a Mongoose feature that runs additional queries to fetch referenced documents and merges them client-side. \$lookup is faster (no round trips), but populate is simpler for typical use. Newer Mongoose versions can internally use \$lookup.

Q106. How does the WiredTiger storage engine work?

WiredTiger (default since 3.2) is MongoDB's storage engine. Key features: (1) Document-level concurrency — multiple writes to different documents in the same collection don't block each other. (2) Snappy compression by default. (3) Two caches: WiredTiger internal cache (default 50% of RAM - 1GB) and OS file system cache. (4) MVCC (Multi-Version Concurrency Control) for snapshots and transactions.

Q107. How would you handle schema migrations in MongoDB?

Since MongoDB is schema-less, you have flexibility: (1) **Lazy migration** — update documents on read; new ones use the new schema. (2) **Eager migration** — run a batch script. (3) Use a **schema_version** field to track each document's version, letting application code handle multiple versions during transition. (4) Tools: migrate-mongo, mongoose schema versioning.

Q108. What are the limitations of MongoDB?

(1) 16MB document size limit. (2) Joins (\$lookup) are slower than SQL joins. (3) Transactions are more expensive than single-doc operations. (4) Memory intensive — best when working set fits in RAM. (5) Eventually consistent reads from secondaries by default. (6) Sharding adds operational complexity. (7) Not ideal for highly relational data with many joins.

Q109. How do you backup a MongoDB database?

(1) **mongodump / mongorestore** — logical backup as BSON files. (2) **File system snapshots** — if storage supports it (LVM, EBS) and journaling is enabled. (3) **Replica set backup** — back up a secondary while it's isolated. (4) **MongoDB Atlas** — managed snapshots and point-in-time recovery. For production, automate and test restores regularly.

Q110. What's the difference between `countDocuments()` and `estimatedDocumentCount()`?

`countDocuments(filter)` — accurate count, runs an aggregation; can be slow on large collections; supports filter. **`estimatedDocumentCount()`** — uses collection metadata; instant but only approximate; doesn't accept a filter. Use estimated when you need a quick total and exact accuracy isn't required.

Coding & Scenario Questions

These are practical coding and scenario-based questions commonly asked in Associate Software Engineer interviews. Practice writing and explaining the code.

Q111. Write an Express middleware that logs request method, URL, and response time.

```
function logger(req, res, next) {
  const start = Date.now();
  res.on('finish', () => {
    const duration = Date.now() - start;
    console.log(
      `${req.method} ${req.originalUrl} ${res.statusCode} ${duration}ms`
    );
  });
  next();
}

app.use(logger);
```

Q112. Implement a Promise-based delay function.

```
function delay(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}

// Usage:
async function run() {
  console.log('Start');
  await delay(1000);
  console.log('1 second later');
}
```

Q113. Convert a callback-style function to a Promise.


```
// Original callback-based API
function readUser(id, callback) {
  db.find(id, (err, user) => callback(err, user));
}

// Promise version
function readUserP(id) {
  return new Promise((resolve, reject) => {
    readUser(id, (err, user) => {
      if (err) reject(err);
      else resolve(user);
    });
  });
}

// Or use util.promisify
const { promisify } = require('util');
const readUserP = promisify(readUser);
```

Q114. Write an async function that runs 3 API calls in parallel and aggregates results.

```
async function fetchAll() {
  try {
    const [user, posts, comments] = await Promise.all([
      fetch('/api/user').then(r => r.json()),
      fetch('/api/posts').then(r => r.json()),
      fetch('/api/comments').then(r => r.json())
    ]);
    return { user, posts, comments };
  } catch (err) {
    console.error('One of the calls failed:', err);
    throw err;
  }
}
```

Q115. Build a simple Express REST endpoint with CRUD operations.

```
const express = require('express');
const router = express.Router();
const User = require('../models/User');

// CREATE
router.post('/', async (req, res, next) => {
  try {
    const user = await User.create(req.body);
    res.status(201).json(user);
  } catch (err) { next(err); }
});

// READ all
router.get('/', async (req, res, next) => {
  try {
    const users = await User.find();
    res.json(users);
  } catch (err) { next(err); }
});

// READ one
router.get('/:id', async (req, res, next) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) return res.status(404).json({ error: 'Not found' });
    res.json(user);
  } catch (err) { next(err); }
});

// UPDATE
router.put('/:id', async (req, res, next) => {
  try {
    const user = await User.findByIdAndUpdate(
      req.params.id, req.body, { new: true, runValidators: true }
    );
    if (!user) return res.status(404).json({ error: 'Not found' });
    res.json(user);
  } catch (err) { next(err); }
});

// DELETE
router.delete('/:id', async (req, res, next) => {
  try {
    await User.findByIdAndDelete(req.params.id);
    res.status(204).end();
  } catch (err) { next(err); }
});

module.exports = router;
```

Q116. Write a MongoDB aggregation to get top 5 customers by total spending.

```
db.orders.aggregate([
  { $match: { status: 'completed' } },
  { $group: {
    _id: '$customerId',
    totalSpent: { $sum: '$total' },
    orderCount: { $sum: 1 }
  }},
  { $sort: { totalSpent: -1 } },
  { $limit: 5 },
  { $lookup: {
    from: 'customers',
    localField: '_id',
    foreignField: '_id',
    as: 'customer'
  }},
  { $unwind: '$customer' },
  { $project: {
    name: '$customer.name',
    email: '$customer.email',
    totalSpent: 1,
    orderCount: 1
  } }
]);
```

Q117. Implement JWT authentication middleware.

```
const jwt = require('jsonwebtoken');

function authenticate(req, res, next) {
  const header = req.headers.authorization;
  if (!header || !header.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Missing token' });
  }
  const token = header.split(' ')[1];
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // { id, role, ... }
    next();
  } catch (err) {
    res.status(401).json({ error: 'Invalid or expired token' });
  }
}

// Usage:
app.get('/profile', authenticate, (req, res) => {
  res.json({ user: req.user });
});
```

Q118. Implement role-based authorization middleware.

```
function requireRole(...allowedRoles) {
  return (req, res, next) => {
    if (!req.user) {
      return res.status(401).json({ error: 'Not authenticated' });
    }
    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Forbidden' });
    }
    next();
  };
}

// Usage: only admin and editor can DELETE
app.delete('/posts/:id',
  authenticate,
  requireRole('admin', 'editor'),
  deletePost
);
```

Q119. Write a Mongoose schema for a blog post with comments embedded.

```
const { Schema, model } = require('mongoose');

const commentSchema = new Schema({
  author: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  text: { type: String, required: true, maxlength: 1000 },
  createdAt: { type: Date, default: Date.now }
});

const postSchema = new Schema({
  title: { type: String, required: true, trim: true },
  slug: { type: String, required: true, unique: true, lowercase: true },
  content: { type: String, required: true },
  author: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  tags: [String],
  comments: [commentSchema],
  published: { type: Boolean, default: false },
  views: { type: Number, default: 0 }
}, { timestamps: true });

postSchema.index({ title: 'text', content: 'text' });
postSchema.index({ tags: 1, createdAt: -1 });

module.exports = model('Post', postSchema);
```

Q120. Implement pagination for an Express + MongoDB endpoint.

```
// GET /api/users?page=2&limit=10
router.get('/', async (req, res, next) => {
  try {
    const page = Math.max(1, parseInt(req.query.page) || 1);
    const limit = Math.min(100, parseInt(req.query.limit) || 10);
    const skip = (page - 1) * limit;

    const [users, total] = await Promise.all([
      User.find().sort({ createdAt: -1 }).skip(skip).limit(limit).lean(),
      User.countDocuments()
    ]);

    res.json({
      data: users,
      pagination: {
        page,
        limit,
        total,
        pages: Math.ceil(total / limit)
      }
    });
  } catch (err) { next(err); }
});
```

Q121. Handle `uncaughtException` and `unhandledRejection` in a Node app.

```
// Graceful handling of fatal errors
process.on('uncaughtException', (err) => {
  console.error('Uncaught Exception:', err);
  // Log to error tracker (Sentry, etc.)
  // Best practice: exit and let process manager restart
  process.exit(1);
});

process.on('unhandledRejection', (reason, promise) => {
  console.error('Unhandled Rejection at:', promise, 'reason:', reason);
  // Treat as uncaughtException in newer Node versions
  process.exit(1);
});

// Graceful shutdown
process.on('SIGTERM', () => {
  console.log('SIGTERM received. Closing server...');
  server.close(() => {
    mongoose.connection.close(false, () => {
      process.exit(0);
    });
  });
});
```

Q122. Implement a `debounce` function in JavaScript.

```
function debounce(fn, delay) {
  let timeoutId;
  return function (...args) {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => fn.apply(this, args), delay);
  };
}

// Usage:
const search = debounce((query) => {
  console.log('Searching for:', query);
}, 300);

// Only the last call within 300ms executes
search('a');
search('ab');
search('abc'); // only this runs after 300ms
```

Q123. Implement a retry mechanism with exponential backoff.

```
async function retryWithBackoff(fn, maxRetries = 5, baseDelay = 1000) {
  let attempt = 0;
  while (attempt < maxRetries) {
    try {
      return await fn();
    } catch (err) {
      attempt++;
      if (attempt >= maxRetries) throw err;
      const delay = baseDelay * Math.pow(2, attempt - 1);
      console.warn(`Attempt ${attempt} failed. Retrying in ${delay}ms`);
      await new Promise(r => setTimeout(r, delay));
    }
  }
}

// Usage:
const data = await retryWithBackoff(
  () => fetch('https://api.example.com/data').then(r => r.json()),
  3,
  500
);
```

Q124. Find duplicate emails in a users collection using aggregation.

```
db.users.aggregate([
  { $group: {
    _id: '$email',
    count: { $sum: 1 },
    ids: { $push: '$_id' }
  }},
  { $match: { count: { $gt: 1 } } },
  { $sort: { count: -1 } }
]);

// Result:
// [
//   { _id: 'duplicate@x.com', count: 3, ids: [id1, id2, id3] },
//   { _id: 'another@x.com', count: 2, ids: [id4, id5] }
// ]
```

Q125. Build a real-time chat backend skeleton with Socket.IO.

```
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');

const app = express();
const server = http.createServer(app);
const io = new Server(server, {
  cors: { origin: '*' }
});

io.on('connection', (socket) => {
  console.log('User connected:', socket.id);

  socket.on('joinRoom', (room) => {
    socket.join(room);
    socket.to(room).emit('system', `${socket.id} joined`);
  });

  socket.on('message', ({ room, text }) => {
    io.to(room).emit('message', {
      from: socket.id, text, at: Date.now()
    });
  });

  socket.on('disconnect', () => {
    console.log('User disconnected:', socket.id);
  });
});

server.listen(3000, () => console.log('Server on :3000'));
```

Final Interview Tips

Before the Interview

- Build a sample full-stack project covering all three technologies and host it on GitHub.
- Be able to walk through the architecture, choices, and trade-offs of your project.
- Practice writing code WITHOUT IDE auto-complete on paper or a whiteboard.
- Review the syntax of common patterns: middleware, Mongoose schemas, aggregation pipelines.
- Read the official docs for Node.js, Express.js, and MongoDB at least once.
- Set up a mock interview with a friend or use platforms like Pramp or interviewing.io.

During the Interview

- **Think out loud** — interviewers want to see your thought process, not just the answer.
- **Clarify requirements** — ask about edge cases, data scale, and constraints first.
- **Start simple** — give a working solution first, then optimize.
- **Discuss trade-offs** — every decision has pros and cons; show you understand them.
- **Admit gaps** — if you don't know something, say so and explain how you'd find out.
- **Ask questions** — about the team, tech stack, and challenges. Shows interest and maturity.

Common Mistakes to Avoid

- Jumping to code without understanding the problem.
- Memorizing answers without understanding — interviewers spot it instantly.
- Ignoring error handling — always discuss what could fail.
- Overcomplicating — start with the simplest correct approach.
- Saying "I don't know" without offering to reason through.
- Forgetting to discuss testing, security, or scalability when relevant.

Topics Often Skipped (But Asked!)

- **HTTP fundamentals** — methods, status codes, headers, cookies, CORS, caching.
- **Database indexing strategy** — explain() and the ESR rule come up a lot.
- **Security basics** — OWASP Top 10, password hashing, JWT pitfalls.
- **Testing** — unit vs integration, mocking, test pyramid.
- **Git workflow** — branching, merging, conflict resolution.

- **Deployment** — Docker, environment variables, CI/CD basics.

IMPORTANT: The key to interview success: deep understanding of fundamentals > broad surface knowledge. Master the basics first — event loop, middleware, indexes, aggregation, async patterns — then build up from there.

Good luck with your interviews! Use this guide as a structured reference, but remember: real understanding comes from building projects, debugging issues, and reading source code. Combine this material with hands-on practice for the best results.